

Autenticação do usuário

15

TÓPICOS ABORDADOS

15.1 PRINCÍPIOS DE AUTENTICAÇÃO DE USUÁRIO REMOTO

Autenticação mútua
Autenticação de mão única

15.2 AUTENTICAÇÃO DE USUÁRIO REMOTO USANDO ENCRIPTAÇÃO SIMÉTRICA

Autenticação mútua
Autenticação de mão única

15.3 KERBEROS

Motivação
Kerberos versão 4
Kerberos versão 5

15.4 AUTENTICAÇÃO DE USUÁRIO REMOTO USANDO ENCRIPTAÇÃO ASSIMÉTRICA

Autenticação mútua
Autenticação de mão única

15.5 GERENCIAMENTO DE IDENTIDADES FEDERADAS

Gerenciamento de identidades
Federação de identidade

15.6 VERIFICAÇÃO DE IDENTIDADE PESSOAL

Modelo do sistema PIV
Documentação do sistema PIV
Credenciais e chaves do sistema PIV
Autenticação

15.7 LEITURA RECOMENDADA

15.8 PRINCIPAIS TERMOS, PERGUNTAS PARA REVISÃO E PROBLEMAS

OBJETIVOS DE APRENDIZAGEM

APÓS ESTUDAR ESTE CAPÍTULO, VOCÊ SERÁ CAPAZ DE:

- ☒ Entender a distinção entre identificação e verificação.
- ☒ Apresentar uma visão geral das técnicas para autenticação do usuário remoto usando encriptação simétrica.
- ☒ Fazer uma apresentação sobre Kerberos.
- ☒ Explicar as diferenças entre as versões 4 e 5 do Kerberos.
- ☒ Descrever o uso do Kerberos em múltiplos domínios.
- ☒ Apresentar uma visão geral das técnicas para autenticação de usuário remoto usando encriptação assimétrica.
- ☒ Compreender a necessidade de um sistema federado de gerenciamento de identidade.
- ☒ Explicar o uso de mecanismos de PIV como parte de um sistema de autenticação de usuário.

"Crachás? Não temos crachás! Não precisamos de crachás! Não tenho que lhe mostrar nenhum crachá nojento!"
— *The Treasure of the Sierra Madre*, 1948

Este capítulo examina algumas das funções de autenticação que foram desenvolvidas para dar suporte à autenticação de usuário baseada em rede. O capítulo começa com uma introdução a alguns dos conceitos e principais considerações para autenticação de usuário em uma rede ou pela Internet. A próxima seção examina os protocolos de autenticação do usuário que contam com a encriptação simétrica. Isso é seguido por uma seção sobre um dos serviços de autenticação mais antigos e ainda mais usados: Kerberos. Em seguida, o capítulo examina os protocolos de autenticação de usuário que contam com encriptação assimétrica. Isso é seguido por uma discussão do protocolo de autenticação de usuário X.509. Por fim, apresentamos o conceito de identidade federada.

15.1 PRINCÍPIOS DE AUTENTICAÇÃO DE USUÁRIO REMOTO

Na maior parte dos contextos de segurança de computadores, a autenticação do usuário é o bloco de montagem fundamental e a principal linha de defesa. A autenticação do usuário é a base para a maioria dos tipos de controle de acesso e para irretratabilidade do usuário. A RFC 4949 (*Internet Security Glossary*) define a autenticação do usuário conforme mostramos a seguir.

Por exemplo, o usuário Alice Toklas poderia ter o identificador de usuário ABTOKLAS. Essa informação precisa ser armazenada em qualquer servidor ou sistema de computador que Alice queira usar e poderia ser conhecido dos administradores do sistema e outros usuários. Um item típico de informação de autenticação com esse ID de usuário é uma senha, que é mantida secreta (conhecida apenas por Alice e pelo sistema). Se ninguém puder obter ou descobrir a senha de Alice, então a combinação do ID de usuário de Alice e a senha permite que os administradores montem as permissões de acesso de Alice e realizem a auditoria de sua atividade. Como o ID de Alice não é secreto, os usuários do sistema podem lhe enviar e-mail, mas como sua senha é secreta, ninguém pode fingir ser Alice.

Autenticação é o processo de verificar uma identidade alegada por ou para uma entidade do sistema. Um processo de autenticação consiste em duas etapas:

- **Etapas de identificação:** apresentar um identificador ao sistema de segurança. (Identificadores devem ser atribuídos com cuidado, pois as identidades autenticadas são a base para outros serviços de segurança, como o serviço do controle de acesso.)
- **Etapas de verificação:** apresentar ou gerar informações de identificação que corroboram o vínculo entre a entidade e o identificador.

Basicamente, a identificação é o meio pelo qual um usuário oferece uma identidade alegada ao sistema; a autenticação do usuário é o meio de estabelecimento da validade da alegação. Observe que a autenticação do usuário é diferente da autenticação da mensagem. Conforme definimos no Capítulo 12, a autenticação da mensagem é um procedimento que permite que as partes que se comunicam verifiquem se o conteúdo de uma mensagem recebida não foi alterado e se a origem é autêntica. Este capítulo trata unicamente da autenticação do usuário.

Existem quatro meios gerais de autenticação da identidade de um usuário, que podem ser usados isoladamente ou em combinação:

- **Algo que o indivíduo sabe:** alguns exemplos são uma senha, um número de identificação pessoal (PIN, do acrônimo em inglês para *personal identification number*) ou respostas a um conjunto de perguntas pré-estipuladas.
- **Algo que o indivíduo possui:** alguns exemplos são chaves criptográficas, cartões de senha eletrônica, *smart cards* e chaves físicas. Esse tipo de autenticador é conhecido como um *token*.

- **Algo que o indivíduo é (biometria estática):** alguns exemplos são reconhecimento por impressão digital, retina e face.
- **Algo que o indivíduo faz (biometria dinâmica):** alguns exemplos são reconhecimento pelo padrão de voz, características de escrita manual e ritmo de digitação.

Todos esses métodos, devidamente implementados e usados, podem oferecer autenticação segura do usuário, porém, cada um tem seus problemas. Um adversário pode ser capaz de adivinhar ou roubar uma senha. De modo semelhante, ele pode ser capaz de forjar ou roubar um token. Um usuário pode se esquecer de uma senha ou perder um token. Além do mais, existe um grande overhead administrativo para gerenciar as informações de senha e token nos sistemas e proteger tais informações nos sistemas. Com relação aos autenticadores biométricos, existem vários problemas, incluindo o tratamento de falsos positivos e falsos negativos, aceitação do usuário, custo e conveniência. Para a autenticação do usuário baseada na rede, os métodos mais importantes envolvem chaves criptográficas e algo que o indivíduo sabe, como uma senha.

Autenticação mútua

Uma área de aplicação importante é a dos protocolos de autenticação mútua. Esses protocolos permitem que as partes se comunicando se satisfaçam mutuamente a respeito da identidade um do outro e troquem chaves de sessão. Esse assunto foi examinado no Capítulo 14. Lá, o foco era a distribuição de chaves. Retornamos a esse assunto aqui para considerar as implicações mais amplas da autenticação.

Existem duas questões centrais ao problema de troca de chave autenticada: confidencialidade e prontidão. Para evitar disfarce e impedir comprometimento das chaves de sessão, informações essenciais de identificação e chave de sessão precisam ser comunicadas de forma encriptada. Isso exige a existência anterior de chaves secretas e públicas que podem ser usadas para essa finalidade. A segunda questão, prontidão, é importante em decorrência da ameaça de replicações de mensagem. Essas replicações, no máximo, poderiam permitir que um oponente comprometesse uma chave de sessão ou personificasse outra parte com sucesso. No mínimo, uma replicação bem-sucedida pode interromper as operações apresentando às partes mensagens que parecem ser genuínas, mas não são.

[GONG93] lista os seguintes exemplos de **ataques de replicação**:

1. O ataque de replicação mais simples é aquele em que o oponente simplesmente copia uma mensagem e a replica mais tarde.
2. Um oponente pode replicar uma estampa de tempo dentro de uma janela de tempo válida. Se o original e a replicação chegarem dentro dessa janela de tempo, esse incidente pode ser registrado em log.
3. Como no exemplo (2), um oponente pode replicar uma mensagem com estampa de tempo dentro da janela de tempo válida mas, além disso, ele suprime a mensagem original. Assim, a repetição não pode ser detectada.
4. Outro ataque envolve uma replicação inversa sem modificação. Essa é uma replicação de volta ao emissor da mensagem. Esse ataque é possível se a encriptação simétrica for usada e o emissor não puder reconhecer facilmente a diferença entre mensagens enviadas e recebidas com base no conteúdo.

Uma técnica para lidar com ataques de replicação é conectar um número sequencial a cada mensagem usada em uma troca de autenticação. Uma nova mensagem é aceita somente se seu número de sequência estiver na ordem correta. A dificuldade com essa técnica é que ela exige que uma parte registre o último número de sequência para cada requerente com quem precisa lidar. Devido a esse overhead, os números de sequência geralmente não são usados para autenticação e troca de chave. Em vez disso, uma das duas técnicas gerais a seguir é utilizada:

- **Estampas de tempo:** a parte A aceita uma mensagem como nova somente se ela tiver uma **estampa de tempo** que, no julgamento de A, é próxima o suficiente do conhecimento de A da hora atual. Essa técnica exige que os relógios entre os diversos participantes estejam sincronizados.
- **Desafio/resposta:** a parte A, esperando uma mensagem nova de B, primeiro envia a B um **nonce** (desafio) e exige que a mensagem subsequente (resposta) recebida de B contenha o valor de nonce correto.

Pode-se argumentar (por exemplo, [LAM92a]) que a técnica de estampa de tempo não deve ser usada para aplicações orientadas a conexão, por causa das dificuldades inerentes a essa técnica. Primeiro, algum

tipo de protocolo é necessário para manter sincronismo entre os diversos relógios de processador. Esse protocolo precisa ser tolerante a falhas, lidar com erros de rede e ser seguro, para enfrentar ataques hosts. Segundo, a oportunidade para um ataque bem-sucedido surgirá se houver uma perda temporária de sincronismo resultante de uma falha no mecanismo de relógio de uma das partes. Por fim, por conta da natureza variável e imprevisível dos atrasos da rede, os relógios distribuídos não podem manter sincronismo preciso. Portanto, qualquer procedimento baseado em estampa de tempo precisa permitir uma janela de tempo suficientemente grande para acomodar atrasos de rede ainda que suficientemente pequenos para minimizar a oportunidade de ataque.

Por outro lado, a técnica de desafio-resposta é inadequada para um tipo de aplicação sem conexão, pois requer o overhead de um *handshake* antes de qualquer transmissão sem conexão, efetivamente negando a principal característica de uma transação sem conexão. Para essas aplicações, contar com algum tipo de servidor de tempo seguro e uma tentativa coerente por cada parte, a fim de manter seus relógios em sincronismo, pode ser a melhor abordagem (por exemplo, [LAM92b]).

Autenticação de mão única

Uma aplicação para a qual a encriptação está crescendo em popularidade é o correio eletrônico (e-mail). A própria natureza do correio eletrônico, e seu principal benefício, é que não é preciso que emissor e receptor estejam on-line ao mesmo tempo. Em vez disso, a mensagem de e-mail é encaminhada à caixa de correio eletrônica do receptor, onde é mantida em buffer até que o receptor esteja disponível para lê-la.

O “envelope” ou cabeçalho da mensagem de e-mail precisa estar às claras, para que a mensagem possa ser entregue pelo protocolo de armazenamento e encaminhamento do e-mail, como o Simple Mail Transfer Protocol (SMTP) ou X.400. Porém, normalmente é desejável que o protocolo de tratamento de correio não exija acesso ao formato de texto claro da mensagem, pois isso exigiria confiar no mecanismo de tratamento de correio. Por conseguinte, a mensagem de e-mail deverá ser encriptada de modo que o sistema de tratamento de correio não esteja de posse da chave de deciptação.

Um segundo requisito é o de **autenticação**. Normalmente, o destinatário deseja alguma garantia de que a mensagem é do emissor alegado.

15.2 AUTENTICAÇÃO DE USUÁRIO REMOTO USANDO ENCRIPTAÇÃO SIMÉTRICA

Autenticação mútua

Conforme discutimos no Capítulo 14, uma hierarquia de dois níveis de chaves de encriptação simétricas pode ser usada para fornecer confidencialidade para a comunicação em um ambiente distribuído. Em geral, essa estratégia envolve o uso de um centro de distribuição de chave (CDC) confiável. Cada parte na rede compartilha uma chave secreta, conhecida como chave mestra, com o CDC. O CDC é responsável por gerar chaves a serem usadas por pouco tempo sobre uma conexão entre duas partes, conhecidas como chaves de sessão, e por distribuir essas chaves usando as chaves mestras para proteger a distribuição. Essa técnica é muito comum. Como um exemplo, examinamos o sistema Kerberos na Seção 15.3. A discussão nesta subseção é relevante para o entendimento dos mecanismos do Kerberos.

A Figura 14.3 ilustra uma proposta apresentada inicialmente por Needham e Schroeder [NEED78] para distribuição de chave secreta usando um CDC que, conforme mencionamos no Capítulo 14, inclui características de autenticação. O protocolo pode ser resumido da seguinte forma:¹

1. $A \rightarrow \text{CDC}: ID_A || ID_B || N_1$
2. $\text{CDC} \rightarrow A: E(K_a, [K_s || ID_B || N_1 || E(K_b, [K_s || ID_A])])$
3. $A \rightarrow B: E(K_b, [K_s || ID_A])$
4. $B \rightarrow A: E(K_s, N_2)$
5. $A \rightarrow B: E(K_s, f(N_2))$

¹ A parte à esquerda do sinal de dois pontos indica o emissor e o receptor; a parte à direita indica o conteúdo da mensagem; o símbolo $||$ indica concatenação.

As chaves secretas K_a e K_b são compartilhadas entre A e o CDC, e B e o CDC, respectivamente. A finalidade do protocolo é distribuir uma chave de sessão K_s com segurança para A e B. A adquire com segurança uma nova chave de sessão na etapa 2. A mensagem na etapa 3 pode ser descriptada, e portanto entendida, apenas por B. A etapa 4 reflete o conhecimento de B quanto a K_s e a etapa 5 garante a B o conhecimento de K_s por A, garantindo a B que essa é uma mensagem nova, por causa do uso do nonce N_2 . Lembre-se, pela nossa discussão no Capítulo 14, que a finalidade das etapas 4 e 5 é impedir um certo tipo de ataque de replicação. Em particular, se um oponente for capaz de capturar a mensagem na etapa 3 e replicá-la, isso poderia de alguma forma interromper as operações em B.

Apesar do *handshake* das etapas 4 e 5, o protocolo ainda é vulnerável a uma forma de ataque de replicação. Suponha que um oponente, X, seja capaz de comprometer uma chave de sessão antiga. Evidentemente, essa é uma ocorrência muito menos provável do que a de um oponente ter simplesmente observado e registrado a etapa 3. Apesar disso, esse é um risco de segurança em potencial. X pode personificar A e enganar B para usar a chave antiga simplesmente replicando a etapa 3. A menos que B se lembre indefinidamente de todas as chaves de sessão anteriores usadas com A, B não poderá determinar que essa é uma replicação. Se X puder interceptar a mensagem de *handshake* na etapa 4, então ele pode personificar a resposta de A na etapa 5. A partir desse ponto, X pode enviar mensagens falsas para B, que aparecem a ele como se viessem de A usando uma chave de sessão autenticada.

Denning [DENN81, DENN82] propõe contornar essa deficiência com uma modificação no protocolo de Needham/Schroeder, que inclui a adição de uma estampa de tempo às etapas 2 e 3. Sua proposta assume que as chaves mestras, K_a e K_b , são seguras, e consiste nas seguintes etapas:

1. $A \rightarrow \text{CDC}$: $ID_A \| ID_B$
2. $\text{CDC} \rightarrow A$: $E(K_a, [K_s \| ID_B \| T \| E(K_b, [K_s \| ID_A \| T])])$
3. $A \rightarrow B$: $E(K_b, [K_s \| ID_A \| T])$
4. $B \rightarrow A$: $E(K_s, N_1)$
5. $A \rightarrow B$: $E(K_s, f(N_1))$

T é uma estampa de tempo que garante a A e B que a chave de sessão acabou de ser gerada. Assim, tanto A quanto B sabem que a distribuição de chave é uma troca nova. A e B podem analisar a atualidade dos dados verificando se

$$|\text{Clock} - T| < \Delta t_1 + \Delta t_2$$

onde Δt_1 é a discrepância normal estimada entre o relógio do CDC e o relógio local (em A ou B) e Δt_2 é o tempo do atraso de rede esperado. Cada nó pode definir seu relógio usando alguma fonte de referência padrão. Como a estampa de tempo T está encriptada usando as chaves mestras seguras, um oponente, mesmo com o conhecimento de uma chave de sessão antiga, não pode ter sucesso, pois uma replicação da etapa 3 será detectada por B como não atual.

Um último ponto: as etapas 4 e 5 não foram incluídas na apresentação original [DENN81], mas sim depois [DENN82]. Essas etapas confirmam o recebimento da chave de sessão em B.

O protocolo de Denning parece oferecer um maior grau de segurança em comparação com o protocolo de Needham/Schroeder. Porém, surge uma nova preocupação: a saber, que esse novo esquema depende de relógios sincronizados pela rede. [GONG92] aponta um risco envolvido. O risco é baseado no fato de que os relógios distribuídos podem perder o sincronismo como resultado de sabotagem ou falhas nos relógios ou no mecanismo de sincronismo.² O problema ocorre quando o relógio de um emissor estiver adiantado em relação ao relógio do destinatário desejado. Nesse caso, um oponente pode interceptar uma mensagem do emissor e replicá-la mais adiante, quando a estampa de tempo na mensagem se tornar atual no local do destinatário. Essa replicação poderia causar resultados inesperados. Gong refere-se a esses ataques como **ataques de supressão-replicação**.

Um modo de impedir ataques de supressão-replicação é impor o requisito de que as partes verifiquem regularmente seus relógios contra o do CDC. A outra alternativa, que evita a necessidade de sincronismo de relógio, é contar com os protocolos de *handshaking* usando nonces. Essa última alternativa não é vulnerável

² Essas coisas podem acontecer e acontecem. Nos últimos anos, chips com defeitos foram usados em diversos computadores e outros sistemas eletrônicos para acompanhar hora e data. Os chips tinham uma tendência a avançar um dia [NEUM90].

a um ataque de supressão-replicação, pois os nonces que o destinatário escolherá no futuro são imprevisíveis ao emissor. O protocolo de Needham/Schroeder conta apenas com nonces, mas, como vimos, possui outras vulnerabilidades.

Em [KEHN92], há uma tentativa de responder aos problemas sobre ataques de supressão-replicação e ao mesmo tempo consertar os problemas no protocolo de Needham/Schroeder. Posteriormente, foi observada uma inconsistência nesse último protocolo, e uma estratégia melhorada foi apresentada em [NEUM93a].³ O protocolo é o seguinte:

1. $A \rightarrow B$: $ID_A || N_a$
2. $B \rightarrow CDC$: $ID_B || N_b || E(K_b, [ID_A || N_a || T_b])$
3. $CDC \rightarrow A$: $E(K_a, [ID_B || N_a || K_s || T_b]) || E(K_b, [ID_A || K_s || T_b]) || N_b$
4. $A \rightarrow B$: $E(K_b, [ID_A || K_s || T_b]) || E(K_s, N_b)$

Vamos acompanhar essa troca passo a passo:

1. A inicia a troca de autenticação gerando um nonce, N_a , e enviando isso mais seu identificador para B em texto claro. Esse nonce será retornado a A em uma mensagem encriptada, que inclui a chave da sessão, garantindo a atualidade dos dados para A.
2. B alerta o CDC de que uma chave de sessão é necessária. Sua mensagem para o CDC inclui seu identificador e um nonce, N_b . Esse nonce será retornado a B em uma mensagem encriptada, que inclui a chave da sessão, garantindo a atualidade de dados para B. A mensagem de B para o CDC também inclui um bloco encriptado com a chave secreta compartilhada por B e o CDC. Esse bloco é usado para instruir o CDC a emitir credenciais a A; o bloco especifica o destinatário intencionado quanto às credenciais, um tempo de expiração sugerido para elas e o nonce recebido de A.
3. O CDC passa para A o nonce de B e um bloco encriptado com a chave secreta que B compartilha com o CDC. O bloco serve como um “ticket” que pode ser usado por A para autenticações subsequentes, conforme veremos. O CDC também envia para A um bloco encriptado com a chave secreta compartilhada por A e o CDC. Esse bloco verifica se B recebeu a mensagem inicial de A (ID_B) e se essa é uma mensagem atual e não uma replicação (N_a), e oferece a A uma chave de sessão (K_s) e o limite de tempo sobre seu uso (T_b).
4. A transmite o ticket para B, junto com o nonce de B, com este último encriptado com a chave da sessão. O ticket oferece a B a chave secreta que é usada para decriptar $E(K_s, N_b)$ para recuperar o nonce. O fato de o nonce de B estar encriptado com a chave de sessão autentica que a mensagem veio de A e não é uma replicação.

Esse protocolo oferece um meio eficaz e seguro para A e B estabelecerem uma sessão com uma chave de sessão segura. Além disso, o protocolo deixa A em posse de uma chave que pode ser usada para autenticação subsequente para B, evitando a necessidade de contatar o servidor de autenticação repetidamente. Suponha que A e B estabeleçam uma sessão usando o protocolo mencionado e depois terminem essa sessão. Posteriormente, mas dentro do limite de tempo estabelecido pelo protocolo, A deseja uma nova sessão com B. O resultado é o seguinte protocolo:

1. $A \rightarrow B$: $E(K_b, [ID_A || K_s || T_b]) || N'_a$
2. $B \rightarrow A$: $N'_b || E(K_s, N'_a)$
3. $A \rightarrow B$: $E(K_s, N'_b)$

Quando B recebe a mensagem na etapa 1, ele verifica se o ticket não expirou. Os nonces recém-gerados N_a e N'_b garantem a cada parte que não existe um ataque de replicação.

Em todo o texto anterior, o tempo especificado em T_b é relativo ao relógio de B. Assim, essa estampa de tempo não exige relógios sincronizados, pois B só verifica estampas de tempo geradas por si mesmo.

³ É realmente difícil acertar essas coisas.

Autenticação de mão única

Usando a encriptação simétrica, o cenário de distribuição descentralizada de chaves, ilustrado na Figura 14.5, é impraticável. Esse esquema exige que o emissor emita uma solicitação para o destinatário desejado, espere uma resposta que inclua uma chave de sessão e somente então envie a mensagem.

Com alguma melhoria, a estrutura do CDC, ilustrada na Figura 14.3, é candidata para correio eletrônico encriptado. Como queremos evitar exigir que o destinatário (B) esteja on-line ao mesmo tempo que o emissor (A), as etapas 4 e 5 precisam ser eliminadas. Para uma mensagem com conteúdo M , a sequência é a seguinte:

1. $A \rightarrow \text{CDC}$: $ID_A || ID_B || N_1$
2. $\text{CDC} \rightarrow A$: $E(K_a, [K_s || ID_B || N_1 || E(K_b, [K_s || ID_A])])$
3. $A \rightarrow B$: $E(K_b, [K_s || ID_A]) || E(K_s, M)$

Essa técnica garante que somente o destinatário intencionado de uma mensagem poderá lê-la. Isso também oferece um nível de autenticação de que o emissor é A. Conforme especificado, o protocolo não protege contra replicações. Alguma medida de defesa poderia ser fornecida incluindo-se uma estampa de tempo com a mensagem. Porém, por conta dos atrasos em potencial no processo de e-mail, essas estampas de tempo podem ter utilidade limitada.

15.3 KERBEROS

Kerberos⁴ é um serviço de autenticação desenvolvido como parte do Projeto Athena no MIT. O problema que o Kerberos resolve é o seguinte. Considere um ambiente distribuído aberto, em que os usuários nas estações de trabalho desejam acessar serviços nos servidores distribuídos pela rede. Gostaríamos que os servidores pudessem restringir o acesso a usuários autorizados e pudessem autenticar as solicitações de serviço. Nesse ambiente, uma estação de trabalho não pode ser confiável para identificar seus usuários corretamente com serviços da rede. Em particular, existem as três ameaças a seguir:

1. Um usuário pode ganhar acesso a determinada estação de trabalho e fingir ser outro usuário operando a partir dessa estação de trabalho.
2. Um usuário pode alterar o endereço de rede de uma estação de trabalho de modo que as solicitações enviadas a partir dela pareçam vir da estação de trabalho personificada.
3. Um usuário pode espionar as trocas e usar um ataque de replicação para entrar em um servidor ou interromper as operações.

Em qualquer um desses casos, um usuário não autorizado pode ser capaz de obter acesso a serviços e dados que ele não está autorizado a acessar. Em vez de se basear em protocolos de autenticação elaborados em cada servidor, Kerberos oferece um servidor de autenticação centralizado, cuja função é autenticar usuários aos servidores e os servidores aos usuários. Diferente da maioria dos outros esquemas de autenticação descritos neste livro, Kerberos conta exclusivamente com a encriptação simétrica, não utilizando encriptação de chave pública.

Duas versões do Kerberos são bastante utilizadas. Implementações da versão 4 [MILL88, STEI88] ainda existem. A versão 5 [KOHL94] corrige algumas das deficiências de segurança da versão 4 e foi emitida como um Internet Standard (RFC 4120 e RFC 4121).⁵

Começamos esta seção com uma rápida discussão da motivação para a técnica do Kerberos. Depois, por conta das complexidades do Kerberos, é melhor começar com uma descrição do protocolo de autenticação utilizado na versão 4. Isso nos permitirá ver a essência da estratégia do Kerberos sem considerar alguns dos detalhes exigidos para lidar com ameaças de segurança sutis. Finalmente, examinamos a versão 5.

⁴ “Em mitologia grega, um cão de muitas cabeças, normalmente três, talvez com um rabo de serpente, o guardião da entrada do Hades.” Em *Dictionary of Subjects and Symbols in Art*, de James Hall, Harper & Row, 1979. Assim como o Kerberos grego tem três cabeças, o Kerberos moderno foi idealizado com três componentes para proteger as portas de uma rede: autenticação, irretratabilidade e auditoria. As duas últimas cabeças nunca foram implementadas.

⁵ As versões de 1 a 3 foram versões internas de desenvolvimento. A versão 4 é o Kerberos “original”

Motivação

Se um conjunto de usuários dispor de computadores pessoais dedicados que não possuem conexão em rede, então os recursos e arquivos de um usuário poderão ser protegidos pela segurança física de cada computador pessoal. Quando esses usuários, em vez disso, são atendidos por um sistema centralizado de tempo compartilhado, este precisa oferecer segurança. O sistema operacional pode impor políticas de controle de acesso com base na identidade do usuário e usar o procedimento de login para identificar os usuários.

Hoje, nenhum desses cenários é típico. Mais comum é uma arquitetura distribuída consistindo em estações de trabalho de usuário dedicadas (clientes) e servidores distribuídos ou centralizados. Nesse ambiente, três técnicas de segurança podem ser identificadas:

1. Contar com cada estação de trabalho de cliente individual para garantir a identidade de seu usuário ou usuários e contar com cada servidor para impor uma política de segurança com base na identificação do usuário (ID).
2. Exigir que os sistemas clientes se autenticuem aos servidores, mas confiar no sistema cliente com relação à identidade de seu usuário.
3. Exigir que o usuário prove sua identidade para cada serviço invocado. Além disso, exigir que os servidores provejam sua identidade aos clientes.

Em um ambiente pequeno e fechado, em que todos os sistemas pertencem e são operados por uma única organização, a primeira ou talvez a segunda estratégia podem ser suficientes.⁶ Mas, em um ambiente mais aberto, em que as conexões de rede com outras máquinas são admitidas, a terceira técnica é necessária para proteger informações de usuário e os recursos abrigados no servidor. Kerberos admite essa terceira técnica. Kerberos assume uma arquitetura cliente/servidor distribuída e emprega um ou mais servidores Kerberos para oferecer um serviço de autenticação.

O primeiro relatório publicado sobre Kerberos [STEI88] listou os seguintes requisitos:

- **Seguro:** um espião da rede não poderá obter a informação necessária para personificar um usuário. Em geral, Kerberos deverá ser forte o suficiente para que um oponente em potencial não o considere o elo mais fraco.
- **Confiável:** para todos os serviços que contam com Kerberos para controle de acesso, a falta de disponibilidade do serviço Kerberos significa falta de disponibilidade dos serviços suportados. Logo, Kerberos deverá ser altamente confiável e, além de empregar uma arquitetura de servidor distribuída, com cada sistema sendo capaz de substituir outro.
- **Transparente:** o ideal é que o usuário não esteja ciente de que a autenticação está ocorrendo, além do requisito da entrada de uma senha.
- **Expansível:** o sistema deverá ser capaz de admitir grandes quantidades de clientes e servidores. Isso sugere uma arquitetura modular, distribuída.

Para dar suporte a esses requisitos, o esquema geral do Kerberos é o de um serviço de autenticação de terceiros confiável, que usa um protocolo baseado naquele proposto por Needham e Schroeder [NEED78], discutido na Seção 15.2. Ele é confiável no sentido de que os clientes e os servidores confiam no Kerberos para mediar mútua autenticação. Supondo que o protocolo Kerberos seja bem projetado, então o serviço de autenticação é seguro se o próprio servidor Kerberos for seguro.⁷

⁶ Porém, até mesmo um ambiente fechado sofre a ameaça de ataque por um funcionário descontente.

⁷ Lembre-se de que a segurança do servidor Kerberos não deve ser assumida automaticamente, mas deve ser protegida com cuidado (por exemplo, em uma sala trancada). É bom lembrar o destino do Kerberos grego, a quem Euristeu ordenou a Hércules que capturasse em seu décimo segundo trabalho: “Hércules encontrou o grande cão em sua corrente e o agarrou pela garganta. Imediatamente, as três cabeças tentaram atacar, e Kerberos chicoteou com seu poderoso rabo. Hércules o agarrou cruelmente, e Kerberos relaxou inconsciente. Euristeu pode ter ficado surpreso ao ver Hércules vivo – quando viu as três cabeças babando e o imenso cão a quem pertenciam, ele muito se assustou, e pulou para a segurança do seu grande jarro de bronze”. Em *The Hamlyn Concise Dictionary of Greek and Roman Mythology*, de Michael Stapleton, Hamlyn, 1982.

Kerberos versão 4

A versão 4 do Kerberos utiliza o DES, em um protocolo um tanto complicado, para oferecer o serviço de autenticação. Vendo o protocolo como um todo, é difícil ver a necessidade dos muitos elementos lá contidos. Portanto, adotamos uma estratégia usada por Bill Bryant do Projeto Athena [BRYA88] e chegamos até o protocolo inteiro examinando primeiro os diversos diálogos hipotéticos. Cada diálogo sucessivo aumenta a complexidade para atacar as vulnerabilidades de segurança reveladas no diálogo anterior.

Depois de examinar o protocolo, vemos alguns outros aspectos da versão 4.

UM DIÁLOGO DE AUTENTICAÇÃO SIMPLES

Em um ambiente de rede desprotegido, qualquer cliente pode solicitar serviço de qualquer servidor. O risco de segurança óbvio é o de personificação. Um oponente pode fingir ser outro cliente e obter privilégios não autorizados nas máquinas servidoras. Para frustrar essa ameaça, os servidores precisam ser capazes de confirmar as identidades dos clientes que solicitam o serviço. Cada servidor pode ter que passar por essa tarefa a cada interação entre cliente/servidor, mas, em um ambiente aberto, isso coloca um peso substancial sobre cada servidor.

Uma alternativa é usar um servidor de autenticação (AS – *Authentication Server*) que conheça as senhas de todos os usuários e as armazene em um banco de dados centralizado. Além disso, o AS compartilha uma chave secreta exclusiva com cada servidor. Essas chaves foram distribuídas fisicamente ou de alguma outra maneira segura. Considere o seguinte diálogo hipotético:

(1) $C \rightarrow AS: ID_C || P_C || ID_V$

(2) $AS \rightarrow C: Ticket$

(3) $C \rightarrow V: ID_C || Ticket$

$Ticket = E(K_v, [ID_C || AD_C || ID_V])$

onde

C = cliente

AS = servidor de autenticação

V = servidor

ID_C = identificador do usuário em C

ID_V = identificador de V

P_C = senha do usuário em C

AD_C = endereço de rede de C

K_v = chave de encriptação secreta compartilhada por AS e V

Nesse cenário, o usuário efetua o logon em uma estação de trabalho e solicita acesso ao servidor V . O módulo cliente C na estação de trabalho do usuário solicita a senha dele e depois envia uma mensagem ao AS , que inclui a ID do usuário, a ID do servidor e a senha do usuário. O AS verifica seu banco de dados para saber se o usuário forneceu a senha correta para essa ID de usuário e se ele tem permissão para acessar o servidor V . Se os dois testes passarem, o AS aceita o usuário como autêntico e agora precisa convencer o servidor de que ele é autêntico. Para fazer isso, o AS cria um **ticket** que contém a ID e o endereço de rede do usuário e a ID do servidor. Esse ticket é encriptado usando a chave secreta compartilhada pelo AS e por esse servidor. Esse ticket é então enviado de volta a C . Como o ticket é encriptado, ele não pode ser alterado por C ou por um oponente.

Com esse ticket, C pode agora solicitar o serviço de V . C envia a mensagem a V contendo a ID de C e o ticket. V decripta o ticket e verifica se a ID do usuário no ticket é a mesma que a ID do usuário não encriptado na mensagem. Se os dois coincidirem, o servidor considera o usuário autenticado e concede o serviço solicitado.

Cada um dos ingredientes da mensagem (3) é significativo. O ticket é encriptado para impedir alteração ou falsificação. A ID do servidor (ID_V) é incluído no ticket, para que o servidor possa verificar se decriptou o ticket corretamente. ID_C é incluído no ticket para indicar que ele foi emitido em favor de C . Por fim, AD_C serve para frustrar a seguinte ameaça. Um oponente poderia capturar o ticket transmitido na mensagem (2), depois usar o nome ID_C e transmitir uma mensagem na forma (3) a partir de outra estação de trabalho. O servidor receberia um ticket válido, que combina com a ID do usuário, e concederia acesso a ele nessa outra estação de trabalho. Para impedir esse ataque, o AS inclui no ticket o endereço de rede do qual veio a solicitação original. Agora, o ticket só é válido se for transmitido da mesma estação de trabalho que solicitou o ticket inicialmente.

UM DIÁLOGO DE AUTENTICAÇÃO MAIS SEGURO

Embora o cenário anterior solucione alguns dos problemas de autenticação em um ambiente de rede aberto, alguns deles permanecem. Dois em particular se destacam. Primeiro, gostaríamos de minimizar o número de vezes que um usuário precisa entrar com uma senha. Suponha que cada ticket só possa ser usado uma vez. Se o usuário C efetuar o logon em uma estação de trabalho de manhã e quiser verificar seu e-mail em um servidor de e-mails, C precisa fornecer uma senha para obter um ticket para o servidor de e-mails. Se C quiser verificar seus e-mails várias vezes durante o dia, cada tentativa irá requerer a reentrada da senha. Podemos melhorar as coisas dizendo que os tickets são reutilizáveis. Para uma única sessão de logon, a estação de trabalho pode armazenar o ticket do servidor de e-mails depois de tê-lo recebido e usado em favor do usuário em múltiplos acessos ao servidor de e-mails.

Porém, sob esse esquema, um usuário continua precisando de um novo ticket para cada serviço diferente. Se um usuário quisesse acessar um servidor de impressão, um servidor de e-mails, um servidor de arquivos e assim por diante, a primeira instância de cada acesso exigiria um novo ticket e, portanto, exigiria que o usuário entrasse com a senha.

O segundo problema é que o cenário anterior envolveu uma transmissão de texto claro da senha [mensagem (1)]. Um espião poderia capturar a senha e usar qualquer serviço acessível à vítima.

Para resolver esses problemas adicionais, introduzimos um esquema para evitar senhas de texto claro e um novo servidor, conhecido como **servidor de concessão de ticket** (TGS, do acrônimo em inglês para *ticket-granting server*). O novo cenário, porém ainda hipotético, é o seguinte:

Uma vez por sessão de logon do usuário:

- (1) $C \rightarrow AS: ID_C || ID_{TGS}$
 (2) $AS \rightarrow C: E(K_c, Ticket_{TGS})$

Uma vez por tipo de serviço:

- (3) $C \rightarrow TGS: ID_C || ID_V || Ticket_{TGS}$
 (4) $TGS \rightarrow C: Ticket_v$

Uma vez por sessão de serviço:

- (5) $C \rightarrow V: ID_C || Ticket_v$
 $Ticket_{TGS} = E(K_{TGS}, [ID_C || AD_C || ID_{TGS} || TS_1 || Lifetime_1])$
 $Ticket_v = E(K_v, [ID_C || AD_C || ID_v || TS_2 || Lifetime_2])$

O novo serviço, TGS, emite tickets aos usuários que foram autenticados no AS. Assim, o usuário primeiro solicita um ticket de concessão de ticket ($Ticket_{TGS}$) a partir do AS. O módulo cliente na estação de trabalho do usuário salva esse ticket. Toda vez que o usuário requer acesso a um novo serviço, o cliente se submete ao TGS, usando o ticket para se autenticar. O TGS, então, concede um ticket para o serviço em particular. O cliente salva cada ticket de concessão de serviço e o utiliza para autenticar seu usuário a um servidor toda vez que um serviço em particular for solicitado. Vamos examinar os detalhes desse esquema:

1. O cliente solicita um ticket de concessão de ticket em favor do usuário enviando sua ID de usuário e senha para o AS, junto com a ID do TGS, indicando uma solicitação para usar o serviço do TGS.
2. O AS responde com um ticket que é encriptado com uma chave que é derivada da senha do usuário (K_c). Quando essa resposta chega no cliente, ele pede a senha do usuário, gera a chave e tenta decriptar a mensagem que chega. Se a senha correta for fornecida, o ticket será recuperado com sucesso.

Como apenas o cliente correto deverá conhecer a senha, apenas o usuário correto poderá recuperar o ticket. Assim, usamos a senha para obter credenciais do Kerberos sem ter que transmitir a senha em texto claro. O próprio ticket consiste na ID e endereço de rede do usuário, e a ID do TGS. Isso corresponde ao primeiro cenário. A ideia é que o cliente possa usar esse ticket para solicitar vários tickets de concessão de serviço. Assim, o ticket de concessão de ticket deve ser reutilizável. Porém, não queremos que um oponente seja capaz de capturar o ticket e usá-lo. Considere o cenário a seguir. Um oponente captura o ticket de login e espera até que o usuário efetue um logoff de sua estação de trabalho. Depois, o oponente ganha acesso a essa estação de trabalho ou configura a sua estação com o mesmo endereço de rede da vítima. O oponente poderia reutilizar o ticket para enganar o TGS. Para impedir isso, o ticket inclui uma estampa de tempo, indicando a data e a hora

em que o ticket foi emitido, e um tempo de vida, indicando o tempo pelo qual o ticket é válido (por exemplo, oito horas). Assim, o cliente agora possui um ticket reutilizável e não precisa incomodar o usuário solicitando uma senha a cada nova solicitação de serviço. Finalmente, observe que o ticket de concessão de ticket é encriptado com uma chave secreta conhecida apenas do AS e do TGS. Isso impede a alteração do ticket. O ticket é re-encriptado com uma chave, com base na senha do usuário. Isso garante que o ticket possa ser recuperado apenas pelo usuário correto, oferecendo a autenticação.

Agora que o cliente tem um ticket de concessão de ticket, o acesso a qualquer servidor pode ser obtido com as etapas 3 e 4:

3. O cliente solicita um ticket de concessão de serviço em favor do usuário. Para essa finalidade, o cliente transmite uma mensagem ao TGS contendo a ID do usuário, a ID do serviço desejado e o ticket de concessão de ticket.
4. O TGS decripta o ticket que recebe usando uma chave compartilhada somente pela AS e TGS (K_{TGS}) e verifica o sucesso da decriptação pela presença de sua ID. Ele verifica para ter certeza de que o tempo de vida não expirou. Depois, compara a ID do usuário e o endereço de rede com a informação que chega, para autenticar o usuário. Se o usuário tiver acesso permitido ao servidor V, o TGS emite um ticket para conceder acesso ao serviço solicitado.

O ticket de concessão de serviço tem a mesma estrutura do ticket de concessão de ticket. Na realidade, como o TGS é um servidor, poderíamos esperar que os mesmos elementos sejam necessários para autenticar um cliente com o TGS e um cliente com um servidor de aplicação. Novamente, o ticket contém uma estampa de tempo e tempo de vida. Se o usuário quiser acessar o mesmo serviço mais tarde, o cliente pode simplesmente usar o ticket de concessão de serviço adquirido anteriormente e não precisa incomodar o usuário com um pedido de senha. Observe que o ticket é encriptado com uma chave secreta (K_V) conhecida apenas do TGS e do servidor, impedindo alteração.

Por fim, com um ticket de concessão de serviço em particular, o cliente pode ter acesso ao serviço correspondente à etapa 5:

5. O cliente solicita acesso ao serviço em favor do usuário. Para essa finalidade, o cliente transmite uma mensagem ao servidor, contendo a ID do usuário e o ticket de concessão de serviço. O servidor autentica usando o conteúdo do ticket.

Esse novo cenário satisfaz os dois requisitos apenas de uma consulta de senha por sessão do usuário e proteção da senha do usuário.

O DIÁLOGO DE AUTENTICAÇÃO DA VERSÃO 4

Embora o cenário anterior melhore a segurança em comparação com a primeira tentativa, permanecem dois problemas adicionais. O núcleo do primeiro é o tempo de vida associado ao ticket de concessão de ticket. Se esse tempo de vida for muito curto (por exemplo, minutos), então o usuário será repetidamente solicitado a informar uma senha. Se o tempo de vida for longo (por exemplo, horas), então um oponente terá maior oportunidade para realizar um ataque de replicação. Um oponente poderia espionar a rede e capturar uma cópia do ticket de concessão de ticket e depois esperar que o usuário legítimo saia do sistema. Depois, o oponente poderia forjar o endereço de rede do usuário legítimo e enviar a mensagem da etapa (3) ao TGS. Isso daria ao oponente acesso ilimitado aos recursos e arquivos disponíveis ao usuário legítimo.

De modo semelhante, se um oponente capturar um ticket de concessão de serviço e o usar antes que ele expire, ele terá acesso ao serviço correspondente.

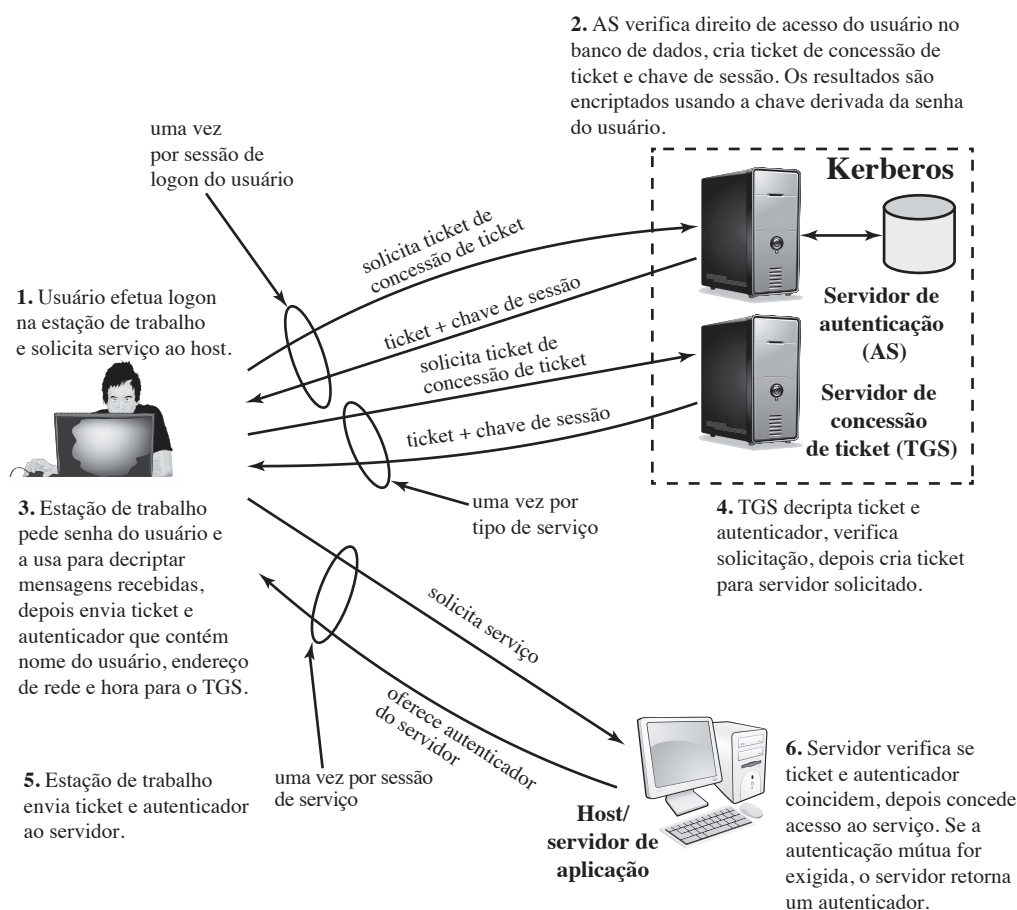
Assim, chegamos a um requisito adicional. Um serviço de rede (o TGS ou um serviço de aplicação) precisa ser capaz de provar que a pessoa usando um ticket é a mesma para quem esse ticket foi emitido.

O segundo problema é que pode haver um requisito para os servidores se autenticarem com os usuários. Sem essa autenticação, um oponente poderia sabotar a configuração de modo que as mensagens para um servidor sejam direcionadas a outro local. O servidor falso, então, estaria em posição de atuar como um servidor real e capturar qualquer informação do usuário e negar o verdadeiro serviço a ele.

Examinamos esses problemas um por vez nos referindo à Tabela 15.1, que mostra o protocolo Kerberos real. A Figura 15.1 oferece uma visão geral simplificada.

Tabela 15.1 Resumo das trocas de mensagem do Kerberos versão 4.

(1) $C \rightarrow AS \quad ID_C ID_{TGS} TS_1$ (2) $AS \rightarrow C \quad E(K_{c, tgs}, [K_{c, v} ID_v TS_2 Lifetime_2 Ticket_{tgs}])$ $Ticket_{tgs} = E(K_{tgs}, [K_{c, tgs} ID_C AD_C ID_{TGS} TS_2 Lifetime_2])$	
(a) Troca de serviço de autenticação para obter ticket de concessão de ticket	
(3) $C \rightarrow TGS \quad ID_v Ticket_{tgs} authenticator_c$ (4) $TGS \rightarrow C \quad E(K_{c, tgs}, [K_{c, v} ID_v TS_4 Ticket_v])$ $Ticket_{tgs} = E(K_{tgs}, [K_{c, tgs} ID_C AD_C ID_{TGS} TS_2 Lifetime_2])$ $Ticket_v = E(K_v, [K_{c, v} ID_C AD_C ID_v TS_4 Lifetime_4])$ $authenticator_c = E(K_{c, tgs}, [ID_C AD_C TS_3])$	
(b) Troca do serviço de concessão de ticket para obter ticket de concessão de serviço	
(5) $C \rightarrow V \quad Ticket_v authenticator_c$ (6) $V \rightarrow C \quad E(K_{c, v}, [TS_5 + 1])$ (para autenticação mútua) $Ticket_v = E(K_v, [K_{c, v} ID_C AD_C ID_v TS_4 Lifetime_4])$ $authenticator_c = E(K_{c, v}, [ID_C AD_C TS_5])$	
(c) Troca de autenticação cliente/servidor para obter serviço	

Figura 15.1 Visão geral do Kerberos.

Primeiro, considere o problema de tickets de concessão de ticket capturados e a necessidade de determinar se o apresentador do ticket é o mesmo cliente para quem o ticket foi emitido. A ameaça é de que um oponente roubará o ticket e o usará antes que expire. Para contornar esse problema, vamos fazer com que o AS ofereça ao cliente e ao TGS uma informação secreta de uma maneira segura. Então, o cliente pode provar sua identidade ao TGS revelando a informação secreta, novamente de uma maneira segura. Um modo eficiente de realizar isso é usar uma chave de encriptação como informação segura; esta é conhecida como chave de sessão no Kerberos.

A Tabela 15.1a mostra a técnica para distribuir a chave de sessão. Como antes, o cliente envia uma mensagem ao AS solicitando acesso ao TGS. O AS responde com uma mensagem, encriptada com uma chave derivada da senha do usuário (K_c), que contém o ticket. A mensagem encriptada também contém uma cópia da chave da sessão, K_{c-tgs} , onde os subscritos indicam que essa é uma chave de sessão para C e TGS. Como essa chave de sessão está dentro da mensagem encriptada com K_c , somente o cliente do usuário poderá lê-la. A mesma chave de sessão está incluída no ticket, que só pode ser lido pelo TGS. Assim, a chave de sessão foi entregue com segurança a C e ao TGS.

Observe que várias informações adicionais foram acrescentadas a essa primeira fase do diálogo. A mensagem (1) inclui uma estampa de tempo, de modo que o AS sabe que ela é recente. A mensagem (2) inclui vários elementos do ticket em uma forma acessível a C. Isso permite que C confirme que esse ticket é para o TGS e descubra sua hora de expiração.

Armado com o ticket e a chave de sessão, C está pronto para abordar o TGS. Como antes, C envia ao TGS uma mensagem que inclui o ticket mais a ID do serviço solicitado [mensagem (3) na Tabela 15.1b]. Além disso, C transmite um autenticador, que inclui a ID e o endereço do usuário de C e uma estampa de tempo. Diferente do ticket, que é reutilizável, o autenticador serve para ser usado apenas uma vez e possui um tempo de vida muito curto. O TGS pode decriptar o ticket com a chave que ele compartilha com o AS. Esse ticket indica que o usuário C recebeu a chave de sessão K_{c-tgs} . Com efeito, o ticket diz: “Qualquer um que use K_{c-tgs} deverá ser C”. O TGS usa a chave de sessão para decriptar o autenticador. O TGS pode, então, verificar o nome e o endereço do autenticador com o do ticket e com o endereço de rede da mensagem que chega. Se tudo coincidir, então o TGS tem garantias de que o emissor do ticket é realmente o proprietário real. Com efeito, o autenticador diz: “No instante TS_3 , eu uso K_{c-tgs} ”. Observe que o ticket não prova a identidade de ninguém, mas é um modo de distribuir chaves com segurança. É o autenticador que prova a identidade do cliente. Como o autenticador só pode ser usado uma vez e tem um tempo de vida curto, a ameaça de um oponente roubar o ticket e o autenticador para apresentação mais adiante é frustrada.

A resposta do TGS, na mensagem (4), segue a forma da mensagem (2). A mensagem é encriptada com a chave de sessão compartilhada pelo TGS e C, e inclui uma chave de sessão a ser compartilhada entre C e o servidor V, a ID de V e a estampa de tempo do ticket. O próprio ticket inclui a mesma chave de sessão.

C agora tem um ticket de concessão de serviço reutilizável para V. Quando C apresenta esse ticket, como vemos na mensagem (5), ele também envia um autenticador. O servidor pode decriptar o ticket, recuperar a chave de sessão e decriptar o autenticador.

Se a autenticação mútua for exigida, o servidor pode responder como mostra a mensagem (6) da Tabela 15.1. O servidor retorna o valor da estampa de tempo a partir do autenticador, incrementado por 1, e encriptado com a chave de sessão. C pode decriptar essa mensagem para recuperar a estampa de tempo incrementada. Como a mensagem foi encriptada pela chave da sessão, C tem garantias de que ela só poderia ter sido criada por V. O conteúdo da mensagem garante a C que essa não é uma replicação de uma resposta antiga.

Por fim, na conclusão desse processo, o cliente e o servidor compartilham uma chave secreta. Essa chave pode ser usada para encriptar mensagens futuras entre os dois ou para trocar uma nova chave de sessão aleatória para essa finalidade.

A Figura 15.2 ilustra as trocas do Kerberos entre as partes. A Tabela 15.2 resume a justificativa para cada um dos elementos no protocolo Kerberos.

Figura 15.2 Trocas do Kerberos.

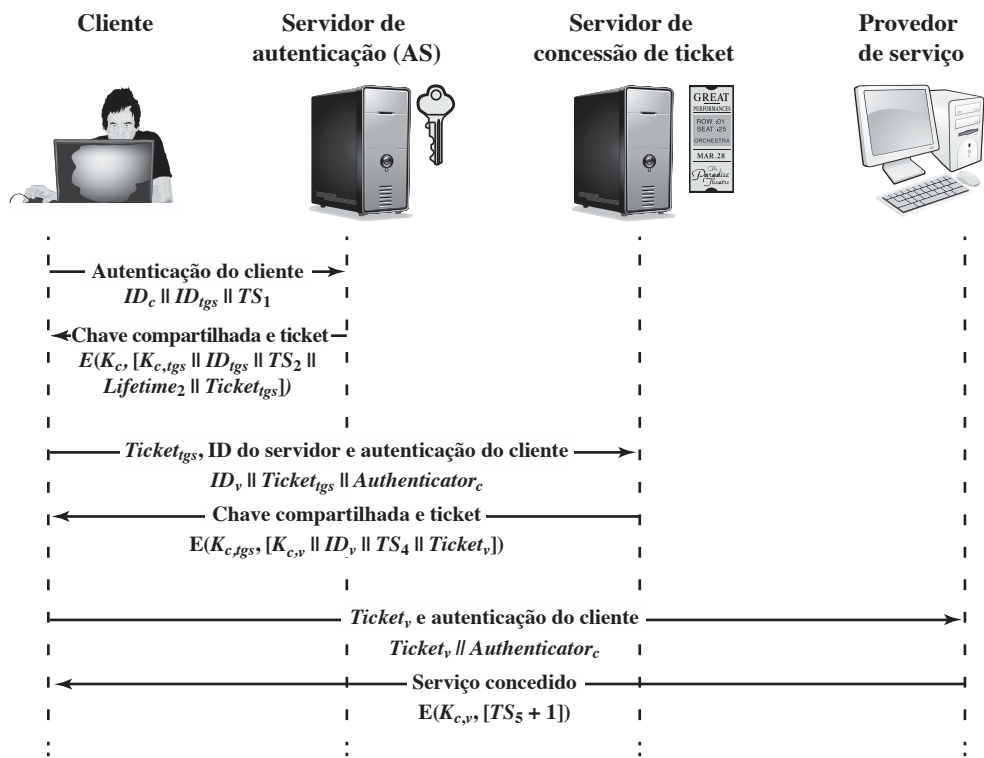


Tabela 15.2 Raciocínio para os elementos do protocolo Kerberos versão 4.

Mensagem (1)	Cliente solicita ticket de concessão de ticket
ID_c	Diz ao AS a identidade do usuário a partir desse cliente
ID_{tgs}	Diz ao AS que o usuário solicita acesso ao TGS
TS_1	Permite que o AS verifique se o relógio desse cliente está sincronizado com o do AS
Mensagem (2)	AS retorna ticket de concessão de ticket
K_c	A encriptação é baseada na senha do usuário, permitindo que o AS e o cliente verifiquem a senha, e protegendo o conteúdo da mensagem (2)
$K_{c,tgs}$	Cópia da chave de sessão acessível ao cliente, criada pelo AS para permitir a troca segura entre cliente e TGS sem exigir que compartilhem uma chave permanente
ID_{tgs}	Confirma que esse ticket é para o TGS
TS_2	Informa ao cliente sobre a hora em que esse ticket foi emitido
$Lifetime_2$	Informa ao cliente sobre o tempo de vida desse ticket
$Ticket_{tgs}$	Ticket a ser usado pelo cliente para acessar o TGS

(a) Trocas no serviço de autenticação

Mensagem (3)	Cliente solicita ticket de concessão de serviço
ID_v	Diz ao TGS que o usuário solicita acesso ao servidor V
$Ticket_{tgs}$	Garante ao TGS que esse usuário foi autenticado pelo AS
$Authenticator_c$	Gerado pelo cliente para validar o ticket
Mensagem (4)	TGS retorna ticket de concessão de serviço
$K_{c,tgs}$	Chave compartilhada apenas por C e TGS protege conteúdo da mensagem (4)
$K_{c,v}$	Cópia da chave de sessão acessível ao cliente, criada pelo TGS para permitir a troca segura entre cliente e servidor, sem exigir que compartilhem uma chave permanente
ID_v	Confirma que esse ticket é para o servidor V

(Continua)

(Continuação)

TS_4	Informa ao cliente sobre a hora em que esse ticket foi emitido
$Ticket_V$	Ticket a ser usado pelo cliente para acessar o servidor V
$Ticket_{tgs}$	Reutilizável de modo que o usuário não precise redigitar a senha
K_{tgs}	O ticket é encriptado com a chave conhecida apenas do AS e do TGS, para impedir modificação
$K_{c,tgs}$	Cópia da chave de sessão acessível ao TGS, usada para decriptar o autenticador, autenticando assim o ticket
ID_C	Indica o proprietário que tem direito a esse ticket
AD_C	Impede o uso de ticket a partir da estação de trabalho que não seja aquela que solicitou o ticket inicialmente
ID_{tgs}	Garante ao servidor que ele decriptou o ticket corretamente
TS_2	Informa ao TGS sobre a hora em que esse ticket foi emitido
$Lifetime_2$	Impede replicação depois que o ticket tiver expirado
$Autenticador_c$	Garante ao TGS que o apresentador do ticket é o mesmo cliente para quem o ticket foi emitido; tem muito pouco tempo de vida, para impedir replicação
$K_{c,tgs}$	Autenticador é encriptado com chave conhecida apenas pelo cliente e pelo TGS, para impedir modificação
ID_C	Precisa coincidir com ID no ticket para autenticá-lo
AD_C	Precisa coincidir com endereço no ticket para autenticá-lo
TS_3	Informa ao TGS sobre a hora em que esse autenticador foi gerado

(b) Trocas no serviço de concessão de ticket

Mensagem (5)	Cliente solicita serviço
$Ticket_V$	Garante ao servidor que esse usuário foi autenticado pelo AS
$Autenticador_c$	Gerado pelo cliente para validar o ticket
Mensagem (6)	Autenticação opcional do servidor ao cliente
$K_{c,v}$	Garante a C que essa mensagem é de V
$TS_5 + 1$	Garante a C que essa não é uma replicação de uma resposta antiga
$Ticket_V$	Reutilizável, de modo que o cliente não precisa solicitar um novo ticket do TGS para cada acesso ao mesmo servidor
K_V	Ticket é encriptado com chave conhecida apenas do TGS e do servidor, para impedir modificação
$K_{c,v}$	Cópia da chave de sessão, acessível ao cliente; usada para decriptar autenticador, autenticando assim o ticket
ID_C	Indica o proprietário que tem direito a esse ticket
AD_C	Impede o uso do ticket a partir de uma estação de trabalho que não seja aquela que solicitou o ticket inicialmente
ID_V	Garante ao servidor que ele decriptou o ticket corretamente
TS_4	Informa ao servidor quanto à hora em que esse ticket foi emitido
$Lifetime_4$	Impede a replicação depois que o ticket tiver expirado
$Autenticador_c$	Garante ao servidor que o apresentador do ticket é o mesmo que o cliente para quem o ticket foi emitido; tem muito pouco tempo de vida, para impedir replicação
$K_{c,v}$	Autenticador é encriptado com chave conhecida apenas do cliente e do servidor, para impedir modificação
ID_C	Precisa combinar com ID no ticket para autenticá-lo
AD_C	Precisa combinar com o endereço no ticket para autenticá-lo
TS_5	Informa ao servidor sobre a hora em que esse autenticador foi gerado

(c) Trocas na autenticação cliente/servidor

DOMÍNIOS DE KERBEROS E KERBERI MÚLTIPLOS

Um ambiente Kerberos de serviço completo, consistindo em um servidor Kerberos, diversos clientes e diversos servidores de aplicação, exige o seguinte:

1. O servidor Kerberos precisa ter as senhas em hash e a ID de todos os usuários participantes em seu banco de dados. Todos os usuários são registrados com o servidor Kerberos.
2. O servidor Kerberos precisa compartilhar uma chave secreta com cada servidor. Todos eles são registrados com o servidor Kerberos.

Esse ambiente é conhecido como **domínio Kerberos**. O conceito de **domínio** pode ser explicado da seguinte forma. Um domínio Kerberos é um conjunto de nós gerenciados que compartilham o mesmo banco de dados Kerberos. Este reside no sistema de computador mestre Kerberos, que deve ser mantido em uma sala fisicamente segura. Uma cópia somente de leitura do banco de dados Kerberos também poderia residir em outros sistemas de computador Kerberos. Porém, todas as mudanças no banco de dados precisam ser feitas no sistema de computador mestre. Para alterar ou acessar o conteúdo de uma base de dados Kerberos, é preciso a senha mestra do Kerberos. Um conceito relacionado é o de um **membro Kerberos**, que é um serviço ou usuário conhecido do sistema Kerberos. Cada membro Kerberos é identificado por seu nome de membro. Estes consistem em três partes: um nome de serviço ou usuário, um nome de instância e um nome de domínio.

Redes de clientes e servidores sob diferentes organizações administrativas normalmente constituem domínios diferentes. Ou seja, geralmente não é prático, ou não está de acordo com a política administrativa, fazer com que usuários e servidores em um domínio administrativo se registrem com um servidor Kerberos em outro lugar. Porém, os usuários em um domínio podem precisar de acesso aos servidores em outros domínios, e alguns servidores podem querer fornecer serviço aos usuários de outros domínios, desde que esses usuários sejam autenticados.

Kerberos oferece um mecanismo para dar suporte a essa autenticação entre domínios. Para dois domínios admitirem a autenticação entre domínios, um terceiro requisito é incluído:

3. O servidor Kerberos em cada domínio interoperante compartilha uma chave secreta com o servidor no outro domínio. Os dois servidores Kerberos são registrados um no outro.

O esquema exige que o servidor Kerberos em um domínio confie no do outro domínio, para autenticar seus usuários. Além disso, os servidores participando do segundo domínio também precisam querer confiar no servidor Kerberos no primeiro.

Com essas regras básicas estabelecidas, podemos descrever o mecanismo da seguinte forma (Figura 15.3): Um usuário desejando serviço em um servidor em outro domínio precisa de um ticket para esse servidor. O cliente do usuário segue os procedimentos normais para ganhar acesso ao TGS local e depois solicita um ticket de concessão de ticket para um TGS remoto (TGS em outro domínio). O cliente pode, então, solicitar do TGS remoto um ticket de concessão de serviço para o servidor desejado no domínio do TGS remoto.

Os detalhes das trocas ilustradas na Figura 15.3 são os seguintes (compare com a Tabela 15.1):

- (1) $C \rightarrow AS: ID_c || ID_{TGS} || TS_1$
- (2) $AS \rightarrow C: E(K_c, [K_{c-TGS} || ID_{TGS} || TS_2 || Lifetime_2 || Ticket_{TGS}])$
- (3) $C \rightarrow TGS: ID_{TGSrem} || Ticket_{TGS} || Authenticator_c$
- (4) $TGS \rightarrow C: E(K_{c-TGS}, [K_{c-TGSrem} || ID_{TGSrem} || TS_4 || Ticket_{TGSrem}])$
- (5) $C \rightarrow TGS_{rem}: ID_{vrem} || Ticket_{TGSrem} || Authenticator_c$
- (6) $TGS_{rem} \rightarrow C: E(K_{c-TGSrem}, [K_{c-vrem} || ID_{vrem} || TS_6 || Ticket_{vrem}])$
- (7) $C \rightarrow V_{rem}: Ticket_{vrem} || Authenticator_c$

O ticket apresentado ao servidor remoto (V_{rem}) indica o domínio em que o usuário foi autenticado originalmente. O servidor escolhe se deve honrar a solicitação remota.

Um problema apresentado pela técnica anterior é que ela não se expande muito bem para muitos domínios. Se houver N domínios, então deverá haver $N(N-1)/2$ trocas de chave seguras, de modo que cada domínio Kerberos possa interoperar com todos os outros.

Kerberos versão 5

Kerberos versão 5 é especificado na RFC 4120 e oferece diversas melhorias em relação à versão 4 [KOHL94]. Para começar, oferecemos uma visão geral das mudanças da versão 4 para a versão 5 e depois examinamos o protocolo da versão 5.

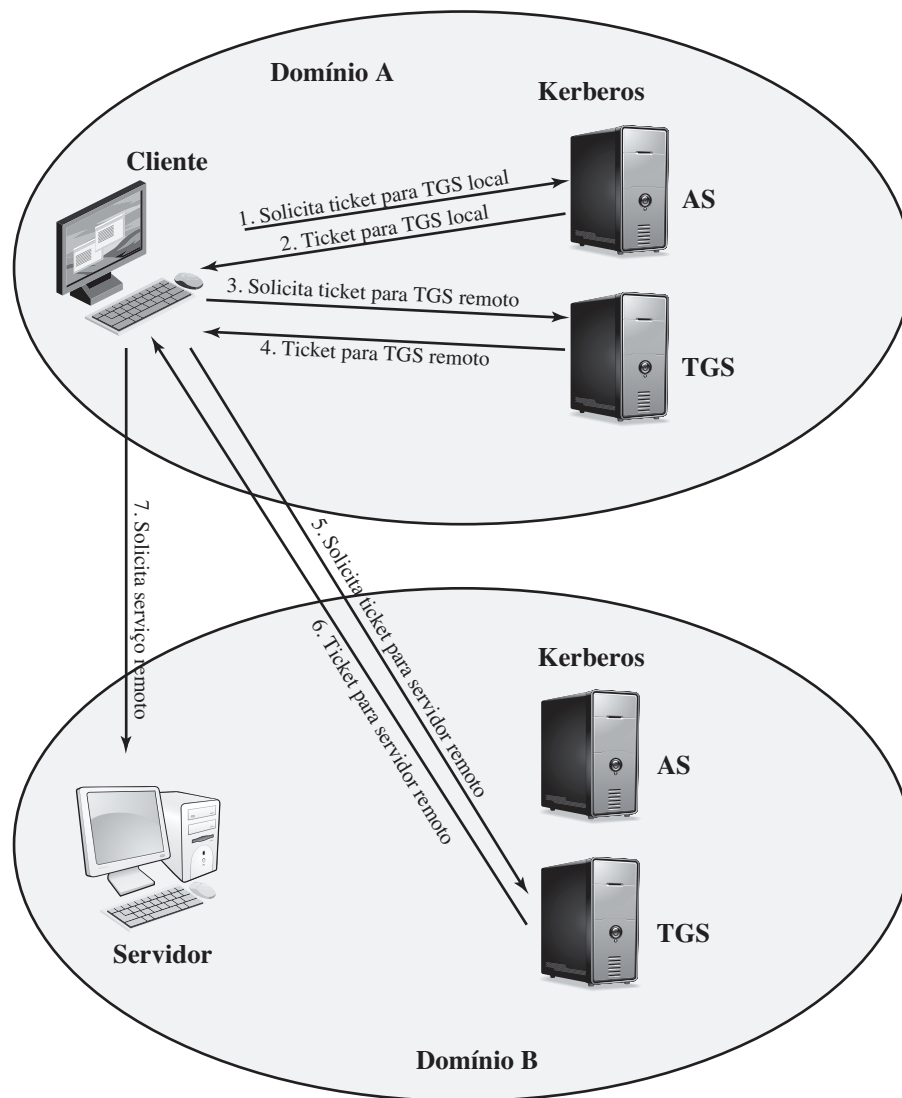
DIFERENÇAS ENTRE AS VERSÕES 4 E 5

A versão 5 tem como objetivo resolver as limitações da versão 4 em duas áreas: limitações ambientais e deficiências técnicas. Vamos resumir rapidamente as melhorias em cada área.⁸

Kerberos versão 4 foi desenvolvido para uso dentro do ambiente do Projeto Athena e, por conseguinte, não atendia totalmente a necessidade de ser de uso geral. Isso levou às seguintes **limitações ambientais**:

⁸ A discussão a seguir segue a apresentação em [KOHL94].

Figura 15.3 Solicitação para serviço em outro domínio.



1. **Dependência do sistema de encriptação:** a versão 4 exige o uso do DES. Assim, a restrição de exportação no DES, além de dúvidas sobre a força do DES, eram uma preocupação. Na versão 5, o texto cifrado é marcado com um identificador de tipo de encriptação para que qualquer técnica de encriptação possa ser usada. As chaves de encriptação são marcadas com um tipo e um tamanho, permitindo que a mesma chave seja usada em diferentes algoritmos e a especificação de diferentes variações sobre determinado algoritmo.
2. **Dependência do protocolo da Internet:** a versão 4 requer o uso de endereços IP (Internet Protocol). Outros tipos de endereço, como o de rede ISO, não são comportados. Os endereços de rede da versão 5 são marcados com tipo e tamanho, permitindo que qualquer tipo de endereço de rede seja utilizado.
3. **Ordenação de byte da mensagem:** na versão 4, o emissor de uma mensagem emprega uma ordenação de byte de sua própria escolha e marca a mensagem para indicar o byte menos significativo no endereço mais baixo ou o byte mais significativo no endereço mais baixo. Essa técnica funciona, mas não segue convenções estabelecidas. Na versão 5, todas as estruturas de mensagem são definidas usando Abstract Syntax Notation One (ASN.1) e Basic Encoding Rules (BER), que oferecem uma ordenação de byte sem ambiguidade.

4. **Tempo de vida do ticket:** os valores de tempo de vida na versão 4 são encriptados em uma quantidade de 8 bits em unidades de cinco minutos. Assim, o tempo de vida máximo que pode ser expresso é $2^8 \times 5 = 1280$ minutos (pouco mais de 21 horas). Isso pode ser inadequado para algumas aplicações (por exemplo, uma simulação de longa duração, que exija credenciais Kerberos válidas durante toda a execução). Na versão 5, os tickets incluem horários inicial e final explícitos, permitindo tickets com quaisquer tempos de vida.
5. **Encaminhamento de autenticação:** a versão 4 não permite que credenciais emitidas para um cliente sejam encaminhadas para algum outro host e usadas por algum outro cliente. Essa capacidade permitiria que um cliente acessasse um servidor e o fizesse acessar outro servidor em favor do cliente. Por exemplo, um cliente emite uma solicitação a um servidor de impressão, que depois acessa o arquivo do cliente a partir de um servidor de arquivos, usando as credenciais do cliente para ter acesso. A versão 5 oferece essa dispendiosa.
6. **Autenticação entre domínios:** na versão 4, a interoperabilidade entre N domínios requer algo na ordem de N^2 relacionamentos Kerberos-para-Kerberos, conforme descrito anteriormente. A versão 5 admite um método que requer menos relacionamentos, conforme explicaremos em breve.

Fora essas limitações ambientais, existem **deficiências técnicas** no próprio protocolo da versão 4. A maioria dessas deficiências foi documentada em [BELL90], e a versão 5 tenta resolvê-las. As deficiências são as seguintes:

1. **Encriptação dupla:** observe na Tabela 15.1 [mensagens (2) e (4)] que os tickets fornecidos aos clientes são encriptados duas vezes, uma com a chave secreta do servidor de destino e depois novamente com uma chave secreta conhecida do cliente. A segunda encriptação não é necessária e é computacionalmente dispendiosa.
2. **Encriptação PCBC:** a encriptação na versão 4 utiliza um modo fora do padrão do DES conhecido como *propagating cipher block chaining* (PCBC).⁹ Tem sido demonstrado que esse modo é vulnerável a um ataque envolvendo o intercâmbio de blocos de texto cifrado [KOHL89]. PCBC foi criado para oferecer uma verificação de integridade como parte da operação de encriptação. A versão 5 oferece mecanismos de integridade explícitos, permitindo que o modo CBC padrão seja usado para encriptação. Em particular, uma soma de verificação ou código de hash é anexado à mensagem antes da encriptação usando CBC.
3. **Chaves de sessão:** cada ticket inclui uma chave de sessão, que é usada pelo cliente para encriptar o autenticador enviado ao serviço associado a esse ticket. Além disso, a chave de sessão pode ser usada posteriormente pelo cliente e o servidor para proteger mensagens passadas durante essa sessão. Porém, como o mesmo ticket pode ser usado repetidamente para obter serviço de um servidor em particular, existe o risco de que um oponente replique mensagens de uma sessão antiga para o cliente ou servidor. Na versão 5, é possível que cliente e servidor negociem uma chave de subsessão, que só deve ser usada para essa única conexão. Um novo acesso pelo cliente resultaria no uso de uma nova chave de subsessão.
4. **Ataques de senha:** as duas versões são vulneráveis a um ataque de senha. A mensagem do AS para o cliente inclui material encriptado com uma chave baseada na senha do cliente.¹⁰ Um oponente pode capturar essa mensagem e tentar decifrá-la experimentando diversas senhas. Se o resultado de uma decifração de teste estiver no formato apropriado, então o oponente terá descoberto a senha do cliente e poderá mais tarde usá-la para obter credenciais de autenticação do Kerberos. Esse é o mesmo tipo de ataque de senha descrito no Capítulo 21, com os mesmos tipos de contramedidas sendo aplicáveis. A versão 5 oferece um mecanismo conhecido como pré-autenticação, que deverá tornar os ataques de senha mais difíceis, mas não os impede.

O DIÁLOGO DE AUTENTICAÇÃO DA VERSÃO 5

A Tabela 15.3 resume o diálogo básico da versão 5. Isso é melhor explicado com uma comparação com a versão 4 (Tabela 15.1).

⁹ Isso será descrito no Apêndice T.

¹⁰ O Apêndice T descreverá o mapeamento entre senhas e chaves de encriptação.

Tabela 15.3 Resumo das trocas de mensagem do Kerberos versão 5.

- $$\begin{aligned}
(1) \quad C \rightarrow AS & \quad Options \parallel ID_c \parallel Realm_c \parallel ID_{TGS} \parallel Times \parallel Nonce_1 \\
(2) \quad AS \rightarrow C & \quad Realm_c \parallel ID_C \parallel Ticket_{TGS} \parallel E(K_{c,TGS}, [K_{c,TGS} \parallel Times \parallel Nonce_1 \parallel Realm_{TGS} \parallel ID_{TGS}]) \\
& \quad Ticket_{TGS} \quad E(K_{TGS}, [Flags \parallel K_{c,TGS} \parallel Realm_c \parallel ID_C \parallel D_C \parallel Times])
\end{aligned}$$

(a) Comunicação do serviço de autenticação para obter ticket de concessão de ticket

- $$\begin{aligned}
(3) \quad C \rightarrow TGS & \quad Options \parallel ID_v \parallel Times \parallel Nonce_2 \parallel Ticket_{TGS} \parallel uthenticator_c \\
(4) \quad TGS \rightarrow C & \quad Realm_c \parallel ID_C \parallel Ticket_v \parallel E(K_{c,TGS}, [K_{c,v} \parallel Times \parallel Nonce_2 \parallel Realm_v \parallel ID_v]) \\
& \quad Ticket_{TGS} \quad E(K_{TGS}, [Flags \parallel K_{c,TGS} \parallel Realm_c \parallel ID_C \parallel D_C \parallel Times]) \\
& \quad Ticket_v \quad E(K_v, [Flags \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel D_C \parallel Times]) \\
& \quad uthenticator_c \quad E(K_{c,TGS}, [ID_C \parallel Realm_c \parallel TS_1])
\end{aligned}$$

(b) Comunicação do serviço de concessão de ticket para obter ticket de concessão de serviço

- $$\begin{aligned}
(5) \quad C \rightarrow V & \quad Options \parallel Ticket_v \parallel uthenticator_c \\
(6) \quad V \rightarrow C & \quad E_{K_{c,v}} [TS_2 \parallel Subkey \parallel Seq \#] \\
& \quad Ticket_v \quad E(K_v, [Flag \parallel K_{c,v} \parallel Realm_c \parallel ID_C \parallel D_C \parallel Times]) \\
& \quad uthenticator_c \quad E(K_{c,v}, [ID_C \parallel Realm_c \parallel TS_2 \parallel Subkey \parallel Seq \#])
\end{aligned}$$

(c) Comunicação de autenticação cliente/servidor para obter serviço

Primeiro, considere a **comunicação do servidor de autenticação**. A mensagem (1) é uma solicitação do cliente para um ticket de concessão de ticket. Como antes, ela inclui o ID do usuário e o TGS. Os novos elementos a seguir são acrescentados:

- **Domínio:** indica o domínio do usuário
- **Opções:** usado para solicitar que certas flags sejam marcadas no ticket retornado
- **Horas:** usado pelo cliente para solicitar as seguintes definições de tempo no ticket:
 - **from:** a hora inicial desejada para o ticket solicitado
 - **till:** a hora de expiração solicitada para o ticket solicitado
 - **rtime:** tempo até a renovação do ticket solicitado
- **Nonce:** um valor aleatório a ser repetido na mensagem (2) para garantir que a resposta é recente e não foi replicada por um oponente.

A mensagem (2) retorna um ticket de concessão de ticket, identificando informação para o cliente, e um bloco encriptado usando a chave de encriptação baseada na senha do usuário. Esse bloco inclui a chave de sessão a ser usada entre o cliente e o TGS, horas especificadas na mensagem (1), o nonce da mensagem (1) e informação de identificação do TGS. O ticket em si inclui a chave de sessão, informação de identificação para o cliente, os valores de hora solicitados e flags que refletem o status desse ticket e das opções solicitadas. Essas flags introduzem nova funcionalidade significativa à versão 5. Por enquanto, adiamos uma discussão dessas flags e nos concentramos na estrutura geral do protocolo da versão 5.

Vamos agora comparar a **comunicação do servidor de concessão de ticket** para as versões 4 e 5. Vemos que a mensagem (3) para as duas versões inclui um autenticador, um ticket e o nome do serviço solicitado. Além disso, a versão 5 inclui os tempos solicitados e opções para o ticket e um nonce, tudo com funções semelhantes àquelas da mensagem (1). O próprio autenticador é basicamente o mesmo que o usado na versão 4.

A mensagem (4) tem a mesma estrutura da mensagem (2), retornando um ticket e informações necessárias pelo cliente, sendo a última encriptada com a chave de sessão agora compartilhada pelo cliente e o TGS.

Por fim, para a **comunicação de autenticação cliente/servidor**, vários recursos novos aparecem na versão 5. Na mensagem (5), o cliente pode solicitar como uma opção que a autenticação mútua seja exigida. O autenticador inclui vários campos novos:

- **Subchave:** a escolha do cliente para uma chave de encriptação a ser usada para proteger a sessão desta aplicação específica. Se esse campo for omitido, a chave de sessão do ticket ($K_{c,v}$) é usada.
- **Número de sequência:** um campo opcional que especifica o número de sequência inicial a ser usado pelo servidor para mensagens enviadas ao cliente durante esta sessão. As mensagens podem ser numeradas em sequência para detectar replicações.

Se for exigida a autenticação mútua, o servidor responde com a mensagem (6). Essa mensagem inclui a estampa de tempo do autenticador. Observe que, na versão 4, a estampa de tempo foi incrementada em um. Isso não é necessário na versão 5, pois a natureza do formato das mensagens é tal que não é possível que um oponente crie a mensagem (6) sem conhecimento das chaves de encriptação apropriadas. O campo de subchave, se estiver presente, substitui o campo de subchave, se estiver presente, na mensagem (5). O campo opcional de número de sequência especifica o número de sequência inicial a ser usado pelo cliente.

FLAGS DO TICKET

O campo de flags incluído nos tickets da versão 5 admite funcionalidade expandida em comparação com o que existe na versão 4. A Tabela 15.4 resume as flags que podem ser incluídas em um ticket.

A flag INITIAL indica que esse ticket foi emitido pelo AS, e não pelo TGS. Quando um cliente solicita um ticket de concessão de serviço do TGS, ele apresenta um ticket de concessão de ticket obtido do AS. Na versão 4, essa era a única maneira de obter um ticket de concessão de serviço. A versão 5 oferece a capacidade adicional de que o cliente pode obter um ticket de concessão de serviço diretamente do AS. A utilidade disso é a seguinte: um servidor, como um de mudança de senha, pode querer saber se a senha do cliente foi testada recentemente.

A flag PRE-AUTHENT, se marcada, indica que, quando o AS recebeu a solicitação inicial [mensagem (1)], ele autenticou o cliente antes de emitir um ticket. A forma exata dessa pré-autenticação não é especificada. Como um exemplo, a implementação do MIT para a versão 5 possui pré-autenticação de estampa de tempo encriptada, ativada como padrão. Quando um usuário deseja obter um ticket, ele precisa enviar ao AS um bloco de pré-autenticação contendo um fator de confusão aleatório, um número de versão e uma estampa de tempo, encriptados com a chave baseada em senha do cliente. O AS decifra o bloco e não enviará um ticket de concessão de ticket a menos que a estampa de tempo no bloco de pré-autenticação esteja dentro do período de tempo permissível

Tabela 15.4 Flags Kerberos versão 5.

INITIAL	Esse ticket foi emitido usando o protocolo AS, e não com base em um ticket de concessão de ticket.
PRE-AUTHENT	Durante a autenticação inicial, o cliente foi autenticado pelo CDC antes que um ticket fosse emitido.
HW-AUTHENT	O protocolo empregado para autenticação inicial exigia o uso de hardware que se esperava que fosse processado unicamente pelo cliente nomeado.
RENEWABLE	Diz ao TGS que esse ticket pode ser usado para obter um substituto que expira em uma data posterior.
MAY-POSTDATE	Diz ao TGS que um ticket pós-datado pode ser emitido com base nesse ticket de concessão de ticket.
POSTDATED	Indica que esse ticket foi pós-datado; o servidor final pode verificar o campo authtime para ver quando ocorreu a autenticação original.
INVALID	Esse ticket é inválido e precisa ser validado pelo CDC antes do uso.
PROXIABLE	Diz ao TGS que um novo ticket de concessão de serviço, com um endereço de rede diferente, pode ser emitido com base no ticket apresentado.
PROXY	Indica que esse ticket é um proxy.
FORWARDABLE	Diz ao TGS que um novo ticket de concessão de ticket, com um endereço de rede diferente, pode ser emitido com base no ticket de concessão de ticket.
FORWARDED	Indica que esse ticket ou foi encaminhado ou foi emitido com base na autenticação envolvendo um ticket de concessão de ticket.

(intervalo de tempo para considerar variação de relógio e atrasos da rede). Outra possibilidade é o uso de um cartão inteligente que gera senhas que mudam continuamente, que são incluídas nas mensagens pré-autenticadas. As senhas geradas pelo cartão podem ser baseadas na senha de um usuário, mas transformadas pelo cartão de modo que, com efeito, sejam utilizadas senhas arbitrárias. Isso impede um ataque baseado em senhas de fácil adivinhação. Se um cartão inteligente ou um dispositivo semelhante foi usado, isso é indicado pela flag HW-AUTHENT.

Quando um ticket tem um tempo de vida longo, existe o potencial de que ele seja roubado e usado por um oponente por um período considerável. Se um tempo de vida curto for usado para diminuir a ameaça, então o overhead é envolvido na aquisição de novos tickets. No caso de um ticket de concessão de ticket, o cliente ou teria que armazenar a chave secreta do usuário, o que certamente é arriscado, ou pedir repetidamente uma senha do usuário. Um esquema de meio termo é o uso de tickets renováveis. Um ticket com a flag RENEWABLE marcada inclui duas horas de expiração: uma para esse ticket específico e uma que é o máximo valor permissível para uma hora de expiração. Um cliente pode ter o ticket renovado apresentando-o ao TGS com uma nova hora de expiração solicitada. Se a nova hora estiver dentro do limite do maior valor permissível, o TGS pode emitir um novo ticket com uma nova hora de sessão e uma nova hora de expiração específica. A vantagem desse mecanismo é que o TGS pode se recusar a renovar um ticket informado como roubado.

Um cliente pode solicitar que o AS ofereça um ticket de concessão de ticket com a flag MAY-POSTDATE marcada. O cliente pode, então, usar esse ticket para solicitar um ticket que é marcado como POSTDATED e INVALID a partir do TGS. Posteriormente, o cliente pode submeter o ticket pós-datado para validação. Esse esquema pode ser útil para executar uma tarefa longa em batch em um servidor que exige um ticket periodicamente. O cliente pode obter diversos tickets para essa sessão de uma só vez, com valores de tempo espalhados. Todos além do primeiro ticket são inicialmente inválidos. Quando a execução chegar ao ponto em que um novo ticket for solicitado, o cliente poderá validar o ticket apropriado. Com essa técnica, o cliente não precisa usar repetidamente seu ticket de concessão de ticket para obter um de concessão de serviço.

Na versão 5, é possível que um servidor atue como um proxy em favor de um cliente, adotando assim as credenciais e os privilégios dele para solicitar um serviço de outro servidor. Se um cliente quiser usar esse mecanismo, ele solicita um ticket de concessão de ticket com a flag PROXIBLE marcada. Quando esse ticket é apresentado ao TGS, este tem permissão para emitir um ticket de concessão de serviço com um endereço de rede diferente; esse último ticket terá sua flag PROXY marcada. Uma aplicação recebendo esse ticket pode aceitá-lo ou exigir autenticação adicional para fornecer uma trilha de auditoria.¹¹

O conceito de proxy é um caso limitado de um procedimento mais poderoso de encaminhamento. Se um ticket for definido com a flag FORWARDABLE, um TGS pode emitir ao solicitante um ticket de concessão de ticket com um endereço de rede diferente e a flag FORWARDED marcada. Esse ticket pode então ser apresentado ao TGS remoto. Essa capacidade permite que um cliente ganhe acesso a um servidor em outro domínio sem exigir que cada Kerberos mantenha uma chave secreta com servidores Kerberos em cada um dos outros domínios. Por exemplo, os domínios poderiam ser estruturados hierarquicamente. Então, um cliente poderia subir na árvore até um nó comum e depois recuar para chegar a um domínio de destino. Cada etapa da caminhada envolveria o encaminhamento de um ticket de concessão de ticket ao próximo TGS no caminho.

15.4 AUTENTICAÇÃO DE USUÁRIO REMOTO USANDO ENCRIPTAÇÃO ASSIMÉTRICA

Autenticação mútua

No Capítulo 14, apresentamos uma técnica para o uso da encriptação de chave pública para fins de distribuição de chave de sessão (Figura 14.9). Esse protocolo considera que cada uma das duas partes está de posse da chave pública atual da outra. Pode não ser prático exigir essa suposição.

Um protocolo usando estampas de tempo aparece em [DENN81]:

1. $A \rightarrow AS: ID_A || ID_B$
2. $AS \rightarrow A: E(PR_{as}, [ID_A || PU_a || T]) || E(PR_{as}, [ID_B || PU_b || T])$
3. $A \rightarrow B: E(PR_{as}, [ID_A || PU_a || T]) || E(PR_{as}, [ID_B || PU_b || T]) || E(PU_b, E(PR_{as}, [K_s || T]))$

¹¹ Para ver uma discussão de alguns dos usos possíveis da capacidade de proxy, consulte [NEUM93b].

Nesse caso, o sistema central é conhecido como um servidor de autenticação (AS), pois não é realmente responsável pela distribuição da chave secreta. Em vez disso, o AS oferece certificados de chave pública. A chave de sessão é escolhida e encriptada por A; logo, não existe risco de exposição pelo AS. As estampas de tempo protegem contra replicações de chaves comprometidas.

Esse protocolo é compacto mas, como antes, exige sincronismo dos relógios. Outra técnica, proposta por Woo e Lam [WOO92a], utiliza nonces. O protocolo consiste nas seguintes etapas:

1. $A \rightarrow CDC:$ $ID_A || ID_B$
2. $CDC \rightarrow A:$ $E(PR_{auth}, [ID_B || PU_b])$
3. $A \rightarrow B:$ $E(PU_b, [N_a || ID_A])$
4. $B \rightarrow CDC:$ $ID_A || ID_B || E(PU_{auth}, N_a)$
5. $CDC \rightarrow B:$ $E(PR_{auth}, [ID_A || PU_a]) || E(PU_b, E(PR_{auth}, [N_a || K_s || ID_B]))$
6. $B \rightarrow A:$ $E(PU_a, [E(PR_{auth}, [(N_a || K_s || ID_B))] || N_b])$
7. $A \rightarrow B:$ $E(K_s, N_b)$

Na etapa 1, A informa ao CDC sobre sua intenção de estabelecer uma conexão segura com B. O CDC retorna a A uma cópia do certificado de chave pública de B (etapa 2). Usando a chave pública de B, A informa a B quanto ao seu desejo de comunicar e envia um nonce N_a (etapa 3). Na etapa 4, B pede ao CDC o certificado de chave pública de A e solicita uma chave de sessão; B inclui o nonce de A de modo que o CDC possa estampar a chave de sessão com esse nonce. O nonce é protegido usando a chave pública do CDC. Na etapa 5, o CDC retorna a B uma cópia do certificado de chave pública de A, mais a informação $\{N_a, K_s, ID_B\}$. Essa informação basicamente diz que K_s é uma chave secreta gerada pelo CDC em favor de B e ligada a N_a ; o vínculo de K_s e N_a garantirá a A que K_s é recente. Essa tripla é encriptada, usando a chave privada do CDC, para permitir que B verifique se a tripla é de fato do CDC. Ela também é encriptada usando a chave pública de B, para que nenhuma outra entidade possa usar a tripla em uma tentativa de estabelecer uma conexão fraudulenta com A. Na etapa 6, a tripla $\{N_a, K_s, ID_B\}$, ainda encriptada com a chave privada do CDC, é repassada para A, junto com um nonce N_b gerado por B. Tudo isso é encriptado usando a chave pública de A. A recupera a chave de sessão K_s e a utiliza para encriptar N_b e retorná-la a B. Essa última mensagem garante a B sobre o conhecimento da chave da sessão por A.

Esse parece ser um protocolo seguro, que leva em conta os diversos ataques. Porém, os próprios autores localizaram uma falha e submeteram uma versão revisada do algoritmo em [WOO92b]:

1. $A \rightarrow CDC:$ $ID_A || ID_B$
2. $CDC \rightarrow A:$ $E(PR_{auth}, [ID_B || PU_b])$
3. $A \rightarrow B:$ $E(PU_b, [N_a || ID_A])$
4. $B \rightarrow CDC:$ $ID_A || ID_B || E(PU_{auth}, N_a)$
5. $CDC \rightarrow B:$ $E(PR_{auth}, [ID_A || PU_a]) || E(PU_b, E(PR_{auth}, [N_a || K_s || ID_A || ID_B]))$
6. $B \rightarrow A:$ $E(PU_a, [N_a || E(PR_{auth}, [(N_a || K_s || ID_A || ID_B))])$
7. $A \rightarrow B:$ $E(K_s, N_b)$

O identificador de A, ID_A , é adicionado ao conjunto de itens encriptados com a chave privada do CDC nas etapas 5 e 6. Isso vincula a chave da sessão K_s às identidades das duas partes que estarão engajadas na sessão. Essa inclusão de ID_A considera o fato de que o valor de nonce N_a é considerado exclusivo somente entre todos os nonces gerados por A, e não entre todos os gerados por todas as partes. Assim, é o par $\{ID_A, N_a\}$ que identifica exclusivamente o pedido de conexão de A.

Neste exemplo e nos protocolos descritos anteriormente, os protocolos que pareciam seguros foram revisados após análise adicional. Esses exemplos realçam a dificuldade de acertar as coisas na área da autenticação.

Autenticação de mão única

Já apresentamos as técnicas de encriptação de chave pública que são adequadas para o correio eletrônico, incluindo a encriptação direta da mensagem inteira para confidencialidade (Figura 12.1b), autenticação (Figura 12.1c) ou ambas (Figura 12.1d). Essas técnicas exigem que o emissor saiba a chave pública do destinatário (confidencialidade) ou o destinatário saiba a chave pública do emissor (autenticação) ou ambos (confidencialidade mais autenticação). Além disso, o algoritmo de chave pública precisa ser aplicado uma ou duas vezes ao que pode ser uma mensagem longa.

Se a confidencialidade for a preocupação principal, então o seguinte poderá ser mais eficiente:

$$A \rightarrow B: E(PU_b, K_s) || E(K_s, M)$$

Nesse caso, a mensagem é encriptada com uma chave secreta de uso único. A também encripta essa chave de uso único com a chave pública de B. Somente B poderá usar a chave privada correspondente para recuperar a chave de uso único e depois usar essa chave para decriptar a mensagem. Esse esquema é mais eficiente do que simplesmente encriptar a mensagem inteira com a chave pública de B.

Se a autenticação for a principal preocupação, então uma assinatura digital poderá ser suficiente, como foi ilustrado na Figura 13.2:

$$A \rightarrow B: M || E(PR_a, H(M))$$

Esse método garante que A não poderá mais tarde negar que enviou a mensagem. Porém, essa técnica está aberta a outro tipo de fraude. Bob redige uma mensagem à sua chefe Alice, que contém uma ideia que economizará dinheiro da empresa. Ele anexa sua assinatura digital e a envia para o sistema de e-mail. Por fim, a mensagem será entregue na caixa de correio de Alice. Mas suponha que Max tenha ouvido falar da ideia de Bob e obtenha acesso à fila de correio antes da entrega. Ele encontra a mensagem de Bob, remove sua assinatura, anexa a dele e reenvia a mensagem para a fila, para ser entregue a Alice. Max recebe o crédito pela ideia de Bob.

Para enfrentar esse esquema, a mensagem e a assinatura podem ser encriptadas com a chave pública do destinatário:

$$A \rightarrow B: E(PU_b, [M || E(PR_a, H(M))])$$

Os dois últimos esquemas exigem que B conheça a chave pública de A e esteja convencido de que ela é recente. Um modo eficiente de fornecer essa garantia é o certificado digital, descrito no Capítulo 14. Agora, temos

$$A \rightarrow B: M || E(PR_a, H(M)) || E(PR_{as}, [T || ID_A || PU_a])$$

Além da mensagem, A envia para B a assinatura encriptada com a chave privada de A, e o certificado de A, encriptado com a chave privada do servidor de autenticação. O destinatário da mensagem primeiro usa o certificado para obter a chave pública do emissor e verificar se ela é autêntica, e depois a usa para verificar a própria mensagem. Se a confidencialidade for exigida, então a mensagem inteira pode ser encriptada com a chave pública de B. Como alternativa, a mensagem inteira pode ser encriptada com uma chave secreta de uso único; a chave secreta também é transmitida, encriptada com a chave pública de B. Essa técnica será explorada no Capítulo 19.

15.5 GERENCIAMENTO DE IDENTIDADES FEDERADAS

O **gerenciamento de identidade federada** é um conceito relativamente novo para lidar com o uso de um esquema comum de gerenciamento de identidade entre diversas empresas e inúmeras aplicações e muitos milhares, até mesmo milhões de usuários. Começamos nossa análise com uma discussão do conceito de gerenciamento de identidades e depois examinamos o gerenciamento de identidades federadas.

Gerenciamento de identidades

O gerenciamento de identidade é um método centralizado, automatizado, para fornecer acesso em nível de empresa aos recursos pelos empregados e outros indivíduos autorizados. O foco do gerenciamento de identidade é definir uma identidade para cada usuário (humano ou processo), associando atributos à identidade, e impondo um meio pelo qual um usuário pode verificá-la. O conceito central de um sistema de gerenciamento de identidade é o uso de Single Sign-On (SSO), ou autenticação integrada. SSO permite que um usuário acesse todos os recursos da rede depois de uma única autenticação.

Os serviços típicos fornecidos por um sistema de gerenciamento de identidade federada incluem os seguintes:

- **Ponto de contato:** inclui autenticação em que um usuário corresponde ao nome de usuário fornecido, e o gerenciamento de sessões de usuário/servidor.
- **Serviços de protocolo SSO:** oferece um serviço de token de segurança independente do fornecedor, para dar suporte a uma autenticação integrada para os servidores federados.

- **Serviços de confiança:** os relacionamentos de federação exigem uma federação baseada em relacionamento de confiança entre os parceiros de negócios. Um relacionamento de confiança é representado pela combinação dos tokens de segurança usados para trocar informações sobre um usuário, a informação criptográfica usada para proteger esses tokens de segurança e, de forma ideal, as regras de mapeamento de identidade aplicadas à informação contida dentro desse token.
- **Serviços de chave:** gerenciamento de chaves e certificados.
- **Serviços de identidade:** serviços que oferecem a interface para armazenamento de dados locais, incluindo registros e bancos de dados de usuários, para o gerenciamento de informações relacionadas à identidade.
- **Autorização:** concessão de acesso a serviços e/ou recursos específicos, com base na autenticação.
- **Provisionamento:** inclui a criação de uma conta em cada sistema alvo para o usuário, inclusão ou registro do usuário nas contas, estabelecimento de direitos de acesso ou credenciais para garantir a privacidade e a integridade dos dados da conta.
- **Gerenciamento:** serviços relacionados à configuração e distribuição em tempo de execução.

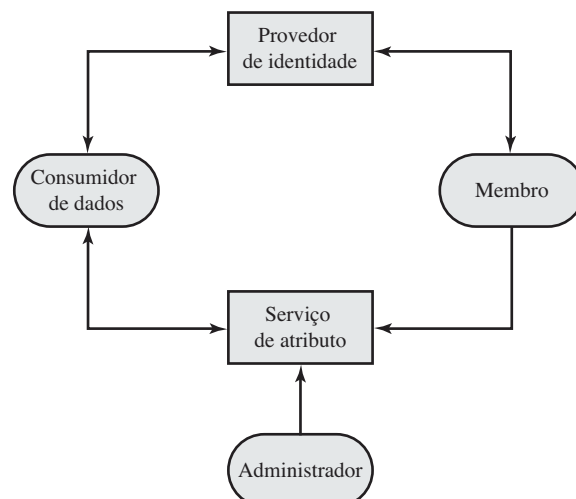
Observe que o Kerberos contém diversos elementos de um sistema de gerenciamento de identidade.

A Figura 15.4 ilustra entidades e fluxos de dados em uma arquitetura genérica de gerenciamento de identidade. Um **membro** é um mantenedor de identidade. Normalmente, esse é um usuário humano que procura acesso a recursos e serviços na rede. Os dispositivos do usuário, processos de agente e sistemas servidores também podem funcionar como membros. Os membros se autenticam com um **provedor de identidade**. Este associa as informações de autenticação a um membro, assim como atributos e um ou mais identificadores.

Cada vez mais, as identidades digitais incorporam atributos além de simplesmente um identificador e informações de autenticação (como senhas e informações biométricas). Um **serviço de atributo** gerencia a criação e a manutenção desses atributos. Por exemplo, um usuário precisa oferecer um endereço de entrega toda vez que um pedido é feito em um novo site de comércio pela Web, e essa informação precisa ser revisada quando o usuário se muda. O gerenciamento de identidade permite que o usuário forneça essa informação uma vez, de modo que ela seja mantida em um único local e liberada aos consumidores de dados de acordo com políticas de autorização e privacidade. Os usuários podem criar alguns dos atributos a serem associados à sua identidade digital, como um endereço. Os **administradores** também podem designar atributos aos usuários, como papéis, permissões de acesso e informações do empregado.

Consumidores de dados são entidades que adquirem e empregam dados mantidos e fornecidos por provedores de identidade e atributo, que geralmente são usados para dar suporte às decisões de autorização e para coletar informações de auditoria. Por exemplo, um servidor de banco de dados ou servidor de arquivos é um consumidor de dados que precisa das credenciais de um cliente para saber que acesso fornecer a esse cliente.

Figura 15.4 Arquitetura genérica do gerenciamento de identidade.



Federação de identidade

A federação de identidade é, basicamente, uma extensão do gerenciamento de identidade para vários domínios de segurança. Esses domínios incluem unidades de negócios internas autônomas, parceiros de negócios externos e outras aplicações e serviços de terceiros. O objetivo é fornecer o compartilhamento de identidades digitais de modo que um usuário possa ser autenticado uma única vez e depois acesse as aplicações e recursos de vários domínios. Como esses domínios são relativamente autônomos ou independentes, não é possível haver qualquer controle centralizado. Em vez disso, as organizações em cooperação precisam formar uma federação com base em padrões combinados e níveis de confiança mútuos para compartilhar identidades digitais com segurança.

O gerenciamento de identidade federada refere-se aos acordos, padrões e tecnologias que permitem a portabilidade de identidades, atributos de identidade e direitos através de várias empresas e diversas aplicações, com suporte para muitos milhares, ou mesmo milhões de usuários. Quando várias organizações implementam esquemas de identidade federada interoperável, um empregado de uma organização pode usar uma autenticação única para acessar serviços pela federação com relacionamentos de confiança associados à identidade. Por exemplo, um empregado pode efetuar o login na intranet de sua empresa e ser autenticado para realizar funções autorizadas e acessar serviços autorizados nessa intranet. O empregado poderia então acessar seus benefícios de saúde a partir de um provedor de saúde externo sem ter que autenticar novamente.

Além do SSO, o gerenciamento de identidade oferece outras capacidades. Uma é um meio padronizado de representar atributos. Cada vez mais, identidades digitais incorporam atributos que não sejam simplesmente um identificador e informações de autenticação (como senhas e informações biométricas). Alguns exemplos de atributos incluem números de conta, papéis organizacionais, localização física e posse de arquivo. Um usuário pode ter vários identificadores; por exemplo, cada um pode estar associado a um papel exclusivo com suas próprias permissões de acesso.

Outra função chave do gerenciamento de identidade federada é o mapeamento de identidade. Diferentes domínios de segurança podem representar identidades e atributos de formas diferentes. Além disso, a quantidade de informação associada a um indivíduo em um domínio pode ser mais do que o necessário em outro domínio. Os protocolos de gerenciamento de identidade federada relacionam identidades e atributos de um usuário em um domínio aos requisitos de outro.

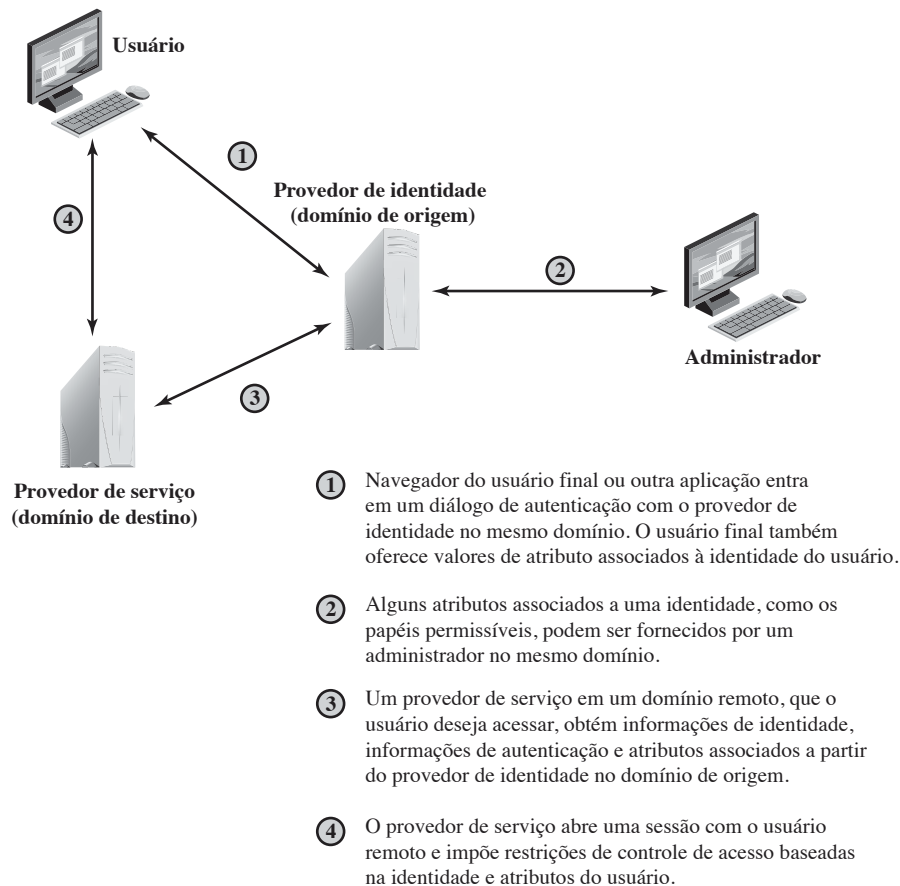
A Figura 15.5 ilustra entidades e fluxos de dados em uma arquitetura genérica de gerenciamento de identidade federada.

O provedor de identidade adquire informações de atributo através do diálogo e trocas de protocolo com usuários e administradores. Por exemplo, um usuário precisa fornecer um endereço de remessa toda vez que um pedido é feito para um comerciante na Web, e essa informação precisa ser revisada quando ele se muda. O gerenciamento de identidade permite que o usuário dê essa informação uma vez, de modo que ela seja mantida em um único local e liberada aos consumidores de dados de acordo com as políticas de autorização e privacidade.

Os provedores de serviço são entidades que obtêm e empregam os dados mantidos e fornecidos pelos provedores de identidade, normalmente para dar suporte a decisões de autorização e para coletar informações de auditoria. Por exemplo, um servidor de banco de dados ou servidor de arquivos é um consumidor de dados que precisa das credenciais de um cliente para saber qual acesso deverá fornecer a esse cliente. Um provedor de serviço pode estar no mesmo domínio do usuário e do provedor de identidade. O poder dessa técnica é para o gerenciamento de identidade federada, no qual o provedor de serviço está em um domínio diferente (por exemplo, uma rede do vendedor ou do fornecedor).

PADRÕES

O gerenciamento de identidade federada usa uma série de padrões como blocos de montagem para a troca de identidade segura por diferentes domínios ou sistemas heterogêneos. Basicamente, as organizações emitem alguma forma de tickets de segurança para seus usuários, que podem ser processados por parceiros em cooperação. Os padrões de federação de identidade, assim, são conectados com a definição desses tickets, em termos do conteúdo e formato, oferecendo protocolos para troca de tickets e realizando uma série de tarefas de gerenciamento. Entre essas tarefas estão a configuração de sistemas para realizar transferências de atributos e mapeamento de identidade, e realização de funções de logging e auditoria. Os principais padrões são os seguintes:

Figura 15.5 Operação de identidade federada.

- **a Extensible Markup Language (XML):** uma linguagem de marcação que usa conjuntos de tags ou etiquetas embutidas para caracterizar os elementos de texto dentro de um documento, a fim de indicar sua aparência, função, significado ou contexto. Documentos XML aparecem de forma semelhante aos HTML (Hypertext Markup Language) que são visíveis como páginas Web, mas possuem maior funcionalidade. XML inclui definições estritas do tipo de dado de cada campo, dando suporte assim a formatos de banco de dados e semântica. XML oferece regras de codificação para comandos que são usados para transferir e atualizar objetos de dados.
- **O Simple Object Access Protocol (SOAP):** um conjunto mínimo de convenções para invocar código usando XML sobre HTTP. Ele permite que as aplicações solicitem serviços umas das outras com solicitações baseadas em XML e recebam respostas como dados formatados com XML. Assim, XML define objetos e estruturas de dados, e SOAP oferece um meio de trocar esses objetos de dados e realizar chamadas de procedimento remoto relacionadas a esses objetos. Veja uma discussão informativa em [ROS06].
- **WS-Security:** um conjunto de extensões SOAP para implementar integridade e confidencialidade de mensagem nos Web services. Para fornecer a troca segura de mensagens SOAP entre as aplicações, WS-Security atribui tokens de segurança a cada mensagem para uso na autenticação.
- **Security Assertion Markup Language (SAML):** uma linguagem baseada em XML para a troca de informações de segurança entre parceiros de negócios on-line. SAML transmite informações de autenticação na forma de asserções sobre sujeitos. Asserções são declarações sobre o sujeito, emitidas por uma entidade autorizada.

O desafio com o gerenciamento de identidade federada é integrar diversas tecnologias, padrões e serviços para fornecer um utilitário seguro e amigável. O segredo disso, como na maioria das áreas de segurança e rede, é contar com alguns padrões amadurecidos bastante aceitos pelo setor. O gerenciamento de identidade federada parece ter alcançado esse nível de maturidade.

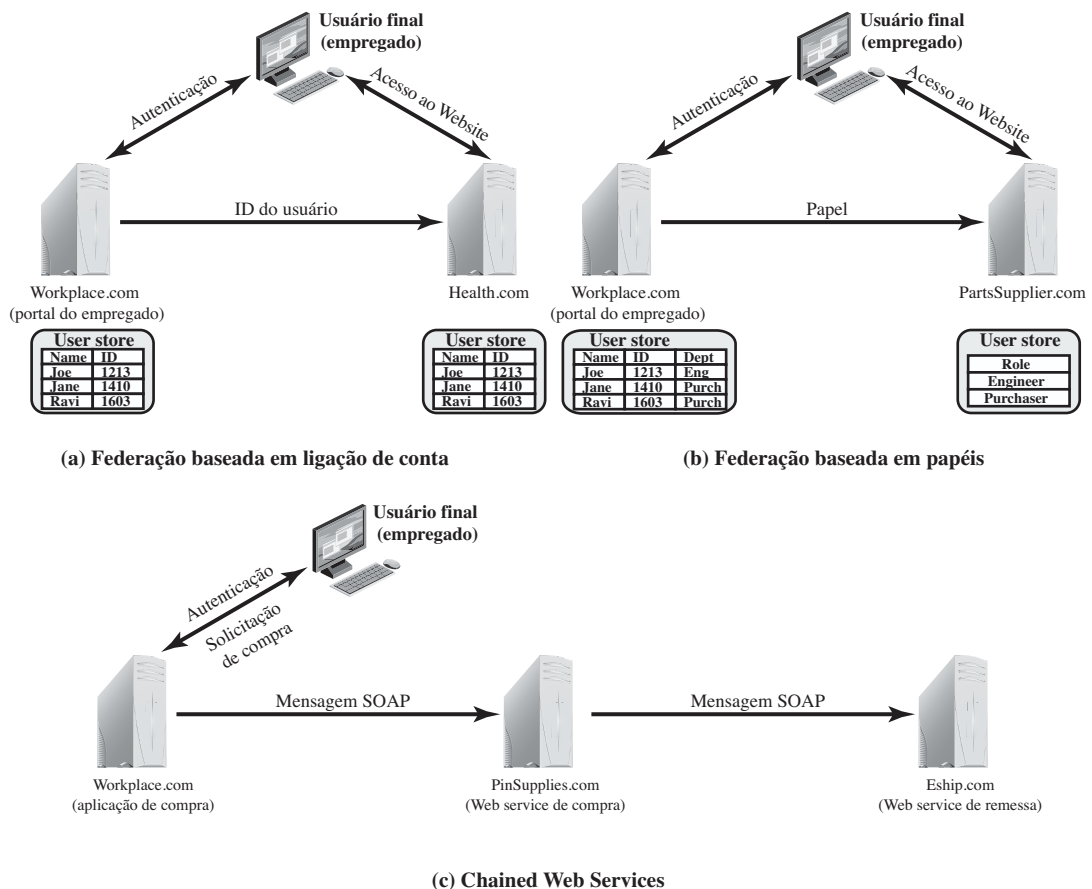
EXEMPLOS

Para ter uma ideia da funcionalidade da federação de identidade, examinamos três cenários, tomados de [COMP06].

No primeiro cenário (Figura 15.6a), Workplace.com contrata Health.com para fornecer benefícios de saúde aos empregados. Um empregado usa uma interface Web para se registrar em Workplace.com e passa por um procedimento de autenticação lá. Isso permite que o empregado acesse serviços e recursos autorizados em Workplace.com. Quando o empregado clica em um link para acessar benefícios de saúde, seu navegador é redirecionado para Health.com. Ao mesmo tempo, o software da Workplace.com passa o identificador do usuário a Health.com de uma maneira segura. As duas organizações fazem parte de uma federação que troca cooperativamente identificadores do usuário. Health.com mantém identidades do usuário para cada empregado em Workplace.com e associa a cada identidade informações de benefícios de saúde e direitos de acesso. Neste exemplo, a ligação entre as duas empresas é baseada na informação de conta e a participação do usuário é baseada no navegador.

A Figura 15.6b mostra um segundo tipo de esquema baseado em navegador. PartsSupplier.com é um fornecedor regular de peças para Workplace.com. Neste caso, um esquema de controle de acesso baseado em papel (RBAC, do acrônimo em inglês para *role-based access-control*) é usado para o acesso à informação. Um engenheiro da Workplace.com se autentica no portal do empregado da Workplace.com e clica em um link para acessar informações em PartsSupplier.com. Como o usuário é autenticado no papel de um engenheiro, ele é levado à parte de documentação técnica e resolução de problemas do Website da PartsSupplier.com sem ter que se autenticar novamente. De modo semelhante, um empregado em um papel de compra se autentica na Workplace.com e está autorizado, nesse papel, a fazer compras na PartsSupplier.com sem ter que se autenticar. Para esse cenário, a PartsSupplier.com não tem informação de identidade para empregados individuais na Workplace.com. Em vez disso, a ligação entre os dois parceiros federados é em termos de papéis.

Figura 15.6 Cenários de identidade federada.



O cenário ilustrado na Figura 15.6c pode ser considerado baseado em documento, em vez de baseado em navegador. Neste terceiro exemplo, Workplace.com tem um acordo de compra com a PinSupplies.com, e esta tem uma relação de negócios com a E-Ship.com. Um empregado da Workplace.com entra e é autenticado para fazer compras. Ele vai para uma aplicação de compras que oferece uma lista de fornecedores da Workplace.com e as peças que podem ser pedidas. O usuário clica no botão da PinSupplies e recebe uma página Web de ordem de compra (página HTML). O empregado preenche o formulário e clica no botão para enviar. A aplicação de compras gera um documento XML/SOAP que ela insere no corpo do envelope de uma mensagem baseada em XML. A aplicação de compras, então, insere as credenciais do usuário no cabeçalho do envelope da mensagem, junto com a identidade organizacional da Workplace.com. A aplicação de compras posta a mensagem no Web service de compras da PinSupplies.com. Esse serviço autentica a mensagem que chega e processa a solicitação. O Web service de compra, então, envia uma mensagem SOAP ao seu parceiro de compras para atender o pedido. A mensagem inclui um token de segurança da PinSupplies.com no cabeçalho do envelope e a lista de itens a serem remetidos, bem como a informação de entrega do usuário final no corpo do envelope. O Web service de entrega autentica a requisição e processa o pedido de envio.

15.6 VERIFICAÇÃO DE IDENTIDADE PESSOAL

A autenticação do usuário baseada na posse de um cartão inteligente (*smart card*) está se tornando mais comum. Um cartão inteligente tem a aparência de um cartão de crédito, uma interface eletrônica e pode usar uma série de protocolos de autenticação.

Um cartão inteligente contém dentro dele um microprocessador inteiro, incluindo processador, memória e portas de E/S. Algumas versões incorporam um circuito de co-processamento especial para a operação criptográfica para agilizar a tarefa de codificar e decodificar mensagens ou gerar assinaturas digitais para validar a informação transferida. Em alguns cartões, as portas de E/S são diretamente acessíveis por uma leitora compatível, por meio de contatos elétricos expostos. Outros cartões, em vez disso, contam com uma antena embutida para a comunicação sem fios com a leitora.

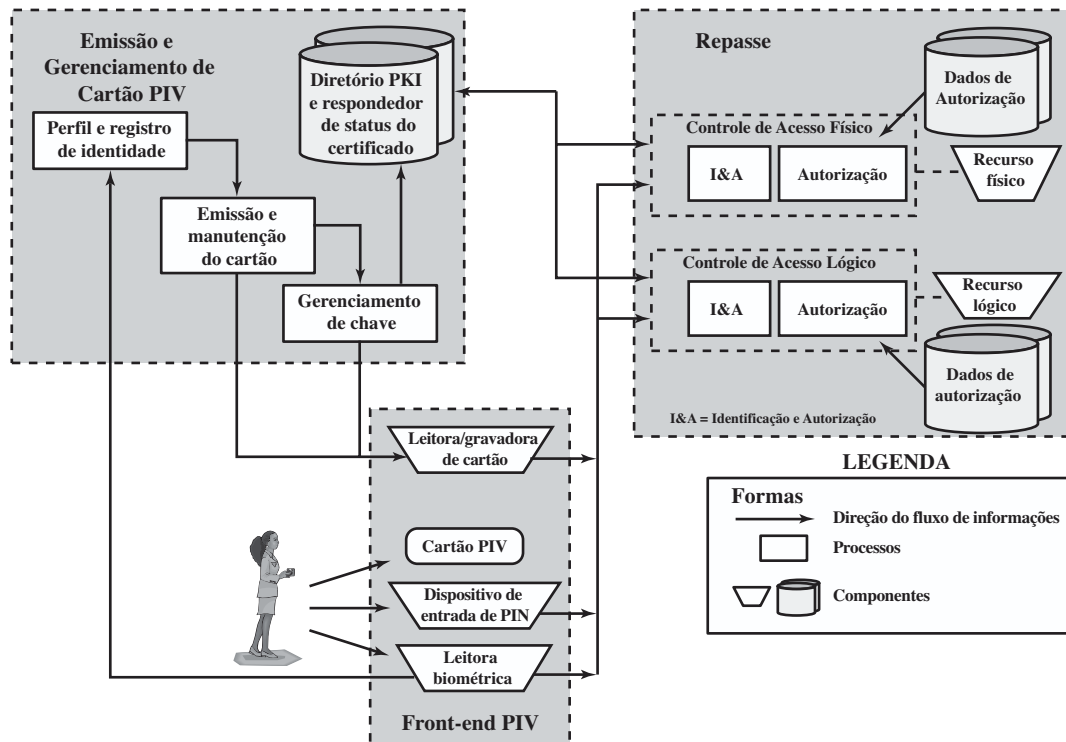
Um cartão inteligente típico inclui três tipos de memória. A memória somente de leitura (ROM, do acrônimo em inglês para *read-only memory*) armazena dados que não mudam durante a vida do cartão, como o número e o nome do seu proprietário. A ROM programável eletricamente apagável (EEPROM, do acrônimo em inglês para *electrically erasable programmable ROM*) mantém dados e programas da aplicação, como os protocolos que o cartão pode executar. Ela também mantém dados que podem variar com o tempo. Por exemplo, em um cartão de telefone, a EEPROM mantém o tempo de conversa restante. A memória de acesso aleatório (RAM, do acrônimo em inglês para *random access memory*) mantém dados temporários gerados quando as aplicações são executadas.

Para uma aplicação prática de autenticação de cartão inteligente, uma grande quantidade de fornecedores precisa estar em conformidade com os padrões que cobrem os protocolos de cartão inteligente, formatos e protocolos de autenticação e controle de acesso, entradas de bancos de dados, formatos de mensagem e assim por diante. Uma etapa importante nessa direção é o FIPS 201-2 (*Personal Identity Verification [PIV] of Federal Employees and Contractors*, junho 2012). O padrão define um sistema de PIV confiável, de âmbito governamental, para uso em aplicações como acesso a instalações e sistemas de informação controlados de modo federado. O padrão especifica um sistema PIV dentro do qual as credenciais de identificação comuns podem ser criadas e mais tarde usadas para verificar uma identidade alegada. O padrão também identifica os requisitos federais em âmbito de governo para níveis de segurança que dependam dos riscos à instalação ou às informações sendo protegidas. O padrão também se aplica a contratantes do setor privado, e serve como uma orientação útil para qualquer organização.

Modelo do sistema PIV

A Figura 15.7 ilustra os principais componentes dos sistemas compatíveis com FIPS 201-2. O front-end PIV define a interface física com um usuário que está solicitando acesso a uma instalação, que poderia ser acesso físico à área física protegida ou acesso lógico a um sistema de informação. O **subsistema de front-end PIV** admite uma autenticação de até três fatores; o número de fatores usados depende do nível de segurança exigido. O front-end utiliza um cartão inteligente, conhecido como cartão PIV, que é um cartão com e sem contato com interface dupla. O cartão contém uma foto do proprietário, certificados X.509, chaves criptográficas, dados

Figura 15.7 Modelo do sistema PIV FIPS 201.



biométricos e um identificador exclusivo do proprietário (CHUID, do acrônimo em inglês para *cardholder unique identifier*). Certas informações do proprietário podem ser protegidas contra leitura e exigem um número de identificação pessoal (PIN, do acrônimo em inglês para *personal identification number*) para o acesso de leitura por parte da leitora de cartões. A leitora biométrica, na versão atual do padrão, é uma leitora de impressão digital ou varredura de íris.

O padrão define três níveis de segurança para verificação do cartão e os dados encriptados armazenados nele, que por sua vez leva à verificação da autenticação da pessoa que mantém a credencial. Um nível de *alguma confiança* corresponde ao uso da leitora de cartão e PIN. Um nível de *alta confiança* adiciona uma comparação biométrica de uma impressão digital capturada e codificada no cartão durante o processo de emissão e uma impressão digital obtida no ponto de acesso físico. Um nível de *confiança muito alta* requer que o processo descrito seja completado em um ponto de controle atendido por um observador oficial.

O outro componente importante do sistema PIV é o **subsistema de emissão e gerenciamento de cartão PIV**. Esse subsistema inclui os componentes responsáveis pela prova e registro de identidade, emissão e gerenciamento de cartão e chave, e diversos repositórios e serviços (por exemplo, diretório de infraestrutura de chave pública [PKI], servidores de status de certificado) exigidos como parte da infraestrutura de verificação.

O sistema PIV interage com um **subsistema de repasse**, que inclui componentes responsáveis por determinar o acesso de determinado proprietário de PIV a um recurso físico ou lógico. FIPS 201-2 padroniza formatos de dados e protocolos para interação entre o sistema PIV e o sistema de repasse.

Diferente do código típico de número de cartão/instalação encriptado na maioria dos cartões de controle de acesso, o CHUID FIPS 201 leva a autenticação a um novo nível, através do uso de uma data de expiração (um campo de dados obrigatório do CHUID) e uma assinatura digital CHUID opcional. Uma assinatura digital pode ser verificada para garantir que o CHUID registrado no cartão foi assinado digitalmente por uma fonte confiável e que os dados CHUID não foram alterados desde que o cartão foi assinado. A data de expiração do CHUID pode ser verificada para garantir que o cartão não foi expirado. Isso depende da data de expiração associada aos privilégios do proprietário do cartão. A leitura e verificação apenas do CHUID oferece apenas alguma garantia de identidade, pois autentica os dados do cartão, e não o proprietário. O PIN e os fatores biométricos oferecem verificação de identidade do indivíduo.

Documentação do sistema PIV

A especificação PIV é bastante complexa, e o NIST emitiu diversos documentos que abrangem uma grande variedade de tópicos sobre PIV. São eles:

- **FIPS 201-2—Personal Identity Verification (PIV) of Federal Employees and Contractors:** especifica as características físicas do cartão, meio de armazenamento e elementos de dados que compõem as credenciais de identidade residentes no cartão PIV.
- **SP 800-73-3—Interfaces for Personal Identity Verification:** especifica as interfaces e a arquitetura de cartão para armazenar e recuperar credenciais de identidade de um cartão inteligente, e oferece diretrizes para o uso dos mecanismos e protocolos de autenticação.
- **SP 800-76-2—Biometric Data Specification for Personal Identity Verification:** descreve as especificações técnicas de aquisição e formatação para as credenciais biométricas do sistema PIV.
- **SP 800-78-3—Cryptographic Algorithms and Key Sizes for Personal Identity Verification:** identifica algoritmos aceitáveis de encriptação simétrica e assimétrica para identificar os algoritmos associados às chaves PIV ou assinaturas digitais.
- **SP 800-104—A Scheme for PIV Visual Card Topography:** oferece recomendações adicionais sobre a codificação de cores de cartão PIV para designar a afiliação do empregado.
- **SP 800-116—A Recommendation for the Use of PIV Credentials in Physical Access Control Systems (PACS):** descreve um método baseado em risco para selecionar mecanismos de autenticação PIV apropriados para gerenciar o acesso físico a instalações e ativos do governo federal.
- **SP 800-79-1—Guidelines for the Accreditation of Personal Identity Verification Card Issuers:** oferece orientações para sancionar a confiabilidade dos emissores de cartões PIV que coletam, armazenam e disseminam credenciais de identidade pessoal e emitem cartões inteligentes.
- **SP 800-96—PIV Card to Reader Interoperability Guidelines:** oferece requisitos que facilitam a interoperabilidade entre qualquer cartão e qualquer leitora.

Além disso, existem outros documentos que lidam com o teste de conformidade e códigos para identificadores.

Credenciais e chaves do sistema PIV

O cartão PIV contém diversos elementos de dados obrigatórios e opcionais que servem como credenciais de identidade com níveis de força e garantia variáveis. Essas credenciais são usadas isoladamente ou em conjuntos para autenticar o proprietário do cartão PIV a fim de conseguir o nível de garantia exigido para determinada atividade ou transação. Os elementos de dados obrigatórios são os seguintes:

- **Número de identificação pessoal (PIN):** exigido para ativar o cartão para operação privilegiada.
- **Identificador exclusivo do proprietário (CHUID):** inclui o Federal Agency Smart Credential Number (FASC-N) e o Global Unique Identification Number (GUID), que identificam exclusivamente o cartão e o proprietário.
- **Chave de identificação PIV:** par de chaves assimétricas e certificado correspondente para autenticação do usuário.
- **Dois modelos de impressão digital:** para autenticação biométrica.
- **Imagem facial eletrônica:** para autenticação biométrica.
- **Chave assimétrica de autenticação de cartão:** par de chaves assimétricas e certificado correspondente usado para autenticação do cartão.

Os elementos opcionais incluem o seguinte:

- **Chave de assinatura digital:** um par de chaves assimétricas e o certificado correspondente, que dá suporte à assinatura de documentos e assinatura de elementos de dados, como o CHUID.
- **Chave de gerenciamento de chave:** um par de chaves assimétricas e o certificado correspondente, dando suporte ao estabelecimento e transporte de chave.
- **Chave simétrica de autenticação de cartão:** para dar suporte a aplicações de acesso físico.
- **Chave de administração da aplicação de cartão PIV:** chave simétrica associada ao sistema de gerenciamento de cartão.
- **Uma ou duas imagens da íris:** para autenticação biométrica.

Tabela 15.5 Algoritmos e tamanhos de chave do sistema PIV.

Tipo de chave PIV	Algoritmos	Tamanhos de chave (bits)	Aplicação
Chave de autenticação PIV	RSA	2048	Suporte para autenticação de cartão e proprietário para um ambiente interoperável
	ECDSA	256	
Chave de autenticação de cartão	3TDEA	168	Suporte para autenticação de cartão para acesso físico
	AES	128, 192 ou 256	
	RSA	2048	Suporte para autenticação de cartão para um ambiente interoperável
	ECDSA	256	
Chave de assinatura digital	RSA	2048 ou 3072	Suporte para assinatura de documento e assinatura de nonce
	ECDSA	256 ou 384	
Chave de gerenciamento de chave	RSA	2048	Suporte para estabelecimento e transporte de chaves
	ECDH	256 ou 384	

A Tabela 15.5 lista o algoritmo e os requisitos de tamanho de chave para os tipos de chave PIV.

Autenticação

Usando as credenciais eletrônicas residentes em um cartão PIV, o cartão tem suporte para os seguintes mecanismos de autenticação:

- **CHUID:** o proprietário é autenticado usando o elemento de dados CHUID assinado no cartão. O PIN não é obrigatório. Esse mecanismo é útil em ambientes onde um baixo nível de segurança é aceitável e a autenticação rápida sem contato é necessária.
- **Chave de autenticação de cartão:** o cartão PIV é autenticado usando a chave de autenticação de cartão em um protocolo desafio-resposta. O PIN não é necessário. Esse mecanismo permite autenticação com contato (via leitora de cartão) ou sem contato (via ondas de rádio) do cartão PIV sem a participação ativa do proprietário, e oferece um baixo nível de segurança.
- **BIO:** o proprietário do cartão é autenticado combinando sua(s) amostra(s) de impressão digital com o elemento de dados biométricos assinado em um ambiente sem um atendente humano à vista. O PIN é necessário para ativar o cartão. Esse mecanismo consegue um alto nível de segurança e requer a participação ativa do proprietário do cartão no envio do PIN e também como amostra biométrica.
- **BIO-A:** o proprietário do cartão é autenticado combinando sua(s) amostra(s) de impressão digital com o elemento de dados biométricos assinado em um ambiente com um atendente humano à vista. O PIN é necessário para ativar o cartão. Esse mecanismo consegue um nível de segurança muito alto quando acoplado com uma validação de confiança completa do modelo biométrico recuperado.

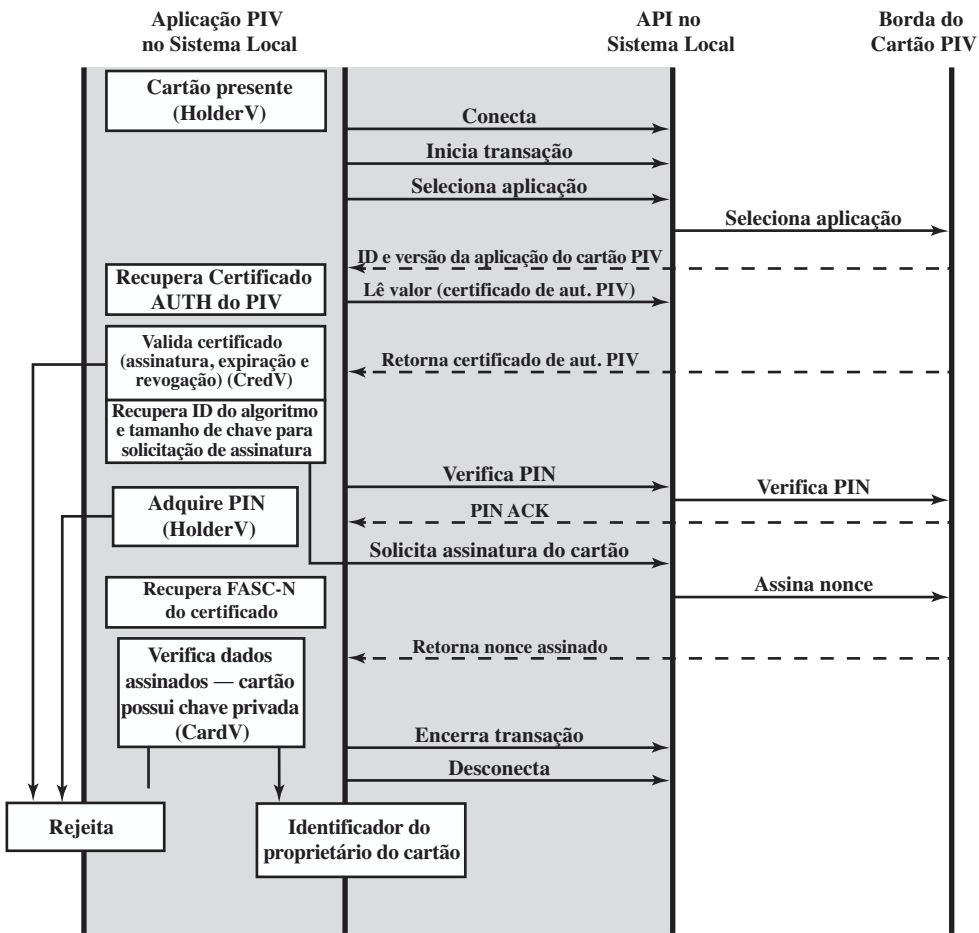
do cartão, e requer a participação ativa do proprietário no envio do PIN e também como amostra biométrica.

- **PKI:** o proprietário do cartão é autenticado demonstrando o controle da chave privada de autenticação do PIV em um protocolo de desafio-resposta que pode ser validado usando o certificado de autenticação PIV. O PIN é necessário para ativar o cartão. Esse mecanismo consegue um nível de segurança de identidade muito alto e requer o conhecimento do PIN pelo proprietário do cartão.

Em cada um dos casos de uso explicados, exceto o da chave de autenticação de cartão simétrica, a origem e a integridade da credencial PIV correspondente são validadas verificando a assinatura digital na credencial, com a assinatura sendo fornecida por uma entidade confiável.

Diversos protocolos podem ser construídos para cada um dos tipos de autenticação. SP-800-78-3 oferece exemplos para cada tipo. A Figura 15.8 ilustra um cenário de autenticação que inclui o uso da chave de autenticação PIV. Esse cenário oferece um alto nível de segurança, e seria apropriado para autenticação de um usuário que possui um cartão PIV e busca acesso a um recurso de computação. O computador, designado *sistema local* na figura, inclui um software de aplicação PIV e se comunica com o cartão por meio de uma interface de programa de aplicação que permite o uso de chamadas de procedimento de nível relativamente alto. Esses comandos de alto nível são convertidos em comandos PIV que são emitidos ao cartão por meio de uma interface física através de uma leitora de cartões ou uma interface sem fios. De qualquer forma, o SP-800-73 refere-se à interface de comando de cartão como a extremidade do cartão PIV.

Figura 15.8 Autenticação usando chave de autenticação PIV.



CardV = Card validation
CredV = Credential validation
HolderV = Cardholder validation
FASC-N = Federal Agency Smart Credential Number

O processo começa quando o sistema local detecta o cartão ou por uma leitora de cartões anexada ou de modo sem fio. Depois, ele seleciona uma aplicação no cartão para autenticação. O sistema local então solicita o certificado de chave pública para a chave de autenticação PIV do cartão. Se o certificado for válido (ou seja, tiver uma assinatura válida, não tiver expirado nem tiver sido revogado), a autenticação continua. Caso contrário, o cartão é rejeitado. A próxima etapa é que o sistema local solicite que o proprietário entre com o PIN para o cartão. Se o PIN submetido coincidir com o PIN armazenado no cartão, este retorna uma confirmação positiva; caso contrário, o cartão retorna uma mensagem de falha. O sistema local ou continua ou rejeita o cartão de modo correspondente. A próxima fase é o protocolo de desafio-resposta. O sistema local envia um nonce a ser assinado pelo PIV, e o PIV retorna a assinatura. O sistema local usa a chave pública de autenticação PIV para verificar a assinatura. Se ela for válida, o proprietário do cartão é aceito como sendo identificado. Caso contrário, o sistema local rejeita o cartão.

O cenário da Figura 15.8 realiza três tipos de autenticação. A combinação de posse do cartão e confirmação do serviço PIN autentica o proprietário do cartão. O certificado de Chave de Autenticação PIV valida as credenciais do cartão. O protocolo de desafio-resposta autentica o cartão.

15.7 LEITURA RECOMENDADA

Uma forma indolor de entender os conceitos do Kerberos pode ser vista em [BRYA88]. Um dos melhores tratamentos do Kerberos é [KOHL94]. [TUNG99] descreve o Kerberos do ponto de vista de um usuário.

[SHIM05] contém uma breve visão geral do gerenciamento de identidade federada e examina uma técnica para padronização. [BHAT07] descreve um método integrado de gerenciamento de identidade federada acoplado com o gerenciamento de privilégios de controle de acesso.

BHAT07 Bhatti, R.; Bertino, E.; e Ghafoor, A. “An Integrated Approach to Federated Identity and Privilege Management in Open Systems”. *Communications of the ACM*, fev 2007.

BRYA88 Bryant, W. Designing an Authentication System: A Dialogue in Four Scenes. Project Athena document, February 1988. Available at <http://web.mit.edu/kerberos/www/dialogue.html>.

KOHL94 Kohl, J.; Neuman, B.; e Ts'o, T. “The Evolution of the Kerberos Authentication Service”. in Brazier, F., and Johansen, D. *Distributed Open Systems*. Los Alamitos, CA: IEEE Computer Society Press, 1994. Disponível em <http://web.mit.edu/kerberos/www/papers.html>.

SHIM05 Shim, S.; Bhalla, G.; e Pendyala, V. “Federated Identity Management”. *Computer*, dez 2005.

TUNG99 Tung, B. *Kerberos: A Network Authentication System*. Reading, MA: Addison-Wesley, 1999.

15.8 PRINCIPAIS TERMOS, PERGUNTAS PARA REVISÃO E PROBLEMAS

Principais termos

ataque de replicação
ataque de supressão-replicação
autenticação
autenticação de mão única
autenticação mútua
domínio

domínio Kerberos
estampa de tempo
gerenciamento de identidade
gerenciamento de identidade federada
Kerberos

nonce
servidor de autenticação
servidor de concessão de ticket (TGS)
ticket
verificação de identidade pessoal (PIV)

Perguntas para revisão

- 15.1 Dê exemplos de ataques de replicação.
- 15.2 Liste três técnicas gerais para lidar com ataques de replicação.
- 15.3 O que é um ataque de supressão-replicação?
- 15.4 Que problema o Kerberos foi criado para resolver?
- 15.5 Quais são três ameaças associadas à autenticação do usuário por uma rede ou pela Internet?
- 15.6 Liste três técnicas para proteger a autenticação do usuário em um ambiente distribuído.
- 15.7 Quais os quatro requisitos definidos para o Kerberos?
- 15.8 Que entidades constituem um ambiente Kerberos com serviço completo?

15.9 No contexto do Kerberos, o que é um domínio?

15.10 Quais são as principais diferenças entre a versão 4 e a versão 5 do Kerberos?

Problemas

15.1 Na Seção 15.4, esboçamos o esquema de chave pública proposto em [WOO92a] para a distribuição de chaves secretas. A versão revisada inclui ID_A nas etapas 5 e 6. Que ataque, especificamente, é evitado por essa revisão?

15.2 O protocolo referenciado no Problema 15.1 pode ser reduzido de sete etapas para cinco, tendo a seguinte sequência:

- a. $A \rightarrow B$:
- b. $A \rightarrow CDC$:
- c. $CDC \rightarrow B$:
- d. $B \rightarrow A$:
- e. $A \rightarrow B$:

Mostre a mensagem transmitida em cada etapa. *Dica:* a mensagem final nesse protocolo é a mesma que a mensagem final no protocolo original.

15.3 Referencie o ataque de supressão-replicação descrito na Seção 15.2 para responder o seguinte:

- a. Dê um exemplo de um ataque quando o relógio de uma parte está adiantado em relação ao do CDC.
- b. Dê um exemplo de um ataque quando o relógio de uma parte está adiantado em relação ao de outra.

15.4 Existem três formas típicas de usar nonces como desafios. Suponha que N_a é um nonce gerado por A, A e B compartilham a chave K e $f()$ é uma função (como um incremento). Os três usos são

Uso 1	Uso 2	Uso 3
(1) $A \rightarrow B: N_a$	(1) $A \rightarrow B: E(K, N_a)$	(1) $A \rightarrow B: E(K, N_a)$
(2) $B \rightarrow A: E(K, N_a)$	(2) $B \rightarrow A: N_a$	(2) $B \rightarrow A: E(K, f(N_a))$

Descreva situações para as quais cada uso é apropriado.

15.5 Mostre que um erro aleatório em um bloco de texto cifrado é propagado para todos os blocos subsequentes de texto claro no modo PCBC (Figura T.2 no Apêndice T).

15.6 Suponha que, no modo PCBC, os blocos C_i e C_{i+1} sejam trocados durante a transmissão. Mostre que isso afeta apenas os blocos decifrados P_i e P_{i+1} , mas não os blocos subsequentes.

15.7 Além de prover um padrão para os formatos de certificado de chave pública, o X.509 especifica um protocolo de autenticação. A versão original do X.509 contém uma falha de segurança. A essência do protocolo é a seguinte:

$A \rightarrow B: A \{t_A, r_A, ID_B\}$

$B \rightarrow A: B \{t_B, r_B, ID_A, r_A\}$

$A \rightarrow B: A \{r_B\}$

onde t_A e t_B são estampas de tempo, r_A e r_B são nonces e a notação $X \{Y\}$ indica que a mensagem Y é transmitida, encriptada e assinada por X.

O texto do X.509 afirma que verificar as estampas de tempo t_A e t_B é opcional para a autenticação de três vias. Mas considere o exemplo a seguir: suponha que A e B tenham usado o protocolo anterior em alguma ocasião anterior, e que o oponente C tenha interceptado as três mensagens anteriores. Além disso, suponha que as estampas de tempo não sejam usadas e sejam todas definidas como 0. Por fim, suponha que C queira personificar A para B. C inicialmente envia a primeira mensagem capturada para B:

$C \rightarrow B: A \{0, r_A, ID_B\}$

B responde pensando que está falando com A, mas na realidade está falando com C:

$B \rightarrow C: B \{0, r'_B, ID_A, r_A\}$

Nesse meio tempo, C faz com que A inicie a autenticação com C por algum meio. Como resultado, A envia o seguinte para C:

$A \rightarrow C: A \{0, r'_A, ID_C\}$

C responde a A usando o mesmo nonce fornecido a C por B.

$$C \rightarrow A: C\{0, r'_B, ID_A, r'_A\}$$

A responde com

$$A \rightarrow C: A\{r'_B\}$$

É exatamente isso que C precisa para convencer B de que está falando com A, de modo que C agora repete a mensagem que chega de volta para B.

$$C \rightarrow B: A\{r'_B\}$$

Assim, B acreditará que está falando com A enquanto na realidade está falando com C. Sugira uma solução simples para esse problema, que não envolva o uso de estampas de tempo.

15.8 Considere uma técnica de autenticação de mão única baseada em encriptação assimétrica:

$$A \rightarrow B: ID_A$$

$$B \rightarrow A: R_1$$

$$A \rightarrow B: E(PR_A, R_1)$$

- a. Explique o protocolo.
- b. A que tipo de ataque esse protocolo é suscetível?

15.9 Considere uma técnica de autenticação de mão única baseada em encriptação assimétrica:

$$A \rightarrow B: ID_A$$

$$B \rightarrow A: E(PU_A, R_2)$$

$$A \rightarrow B: R_2$$

- a. Explique o protocolo.
- b. A que tipo de ataque esse protocolo é suscetível?

15.10 No Kerberos, quando Bob recebe um Ticket de Alice, como ele sabe que ele é genuíno?

15.11 No Kerberos, quando Bob recebe um Ticket de Alice, como ele sabe que ele veio de Alice?

15.12 No Kerberos, quando Alice recebe uma resposta, como ela sabe que ela veio de Bob (que não é uma replicação de uma mensagem anterior de Bob)?

15.13 No Kerberos, o que o Ticket contém que permite que Alice e Bob conversem com segurança?